



Direction de la Météorologie Nationale

École Hassania des Travaux Publics

Approches manuelles et automatiques de segmentation d'images satellitaires pour la détection des zones de convection profonde

Rapport de stage d'ingénieur à DGM

Réalise par : MSAADI ABD-ELLAH
Encadré par : Dr. Badreddine Alaoui
Filière : Génie Météorologie

Année Universitaire 2024-2025

Remerciements

À l'issue de ce stage d'ingénieur, nous souhaitons exprimer nos sincères remerciements à toutes les personnes qui ont contribué à la réussite de cette expérience formatrice. Réalisé au sein de **la Direction Générale de la Météorologie** (DGM), ce stage a représenté une occasion privilégiée d'approfondir nos connaissances et de développer des compétences pratiques dans un cadre stimulant et enrichissant.

Nous tenons à exprimer toute notre reconnaissance à **Dr. Alaoui Badreddine**, Data Scientist et Consultant en Python et R au **Centre de Recherche Météorologique**, pour son accompagnement constant, sa disponibilité et ses conseils éclairés. Son expertise et son encadrement bienveillant ont largement contribué à l'avancement de ce travail et à notre progression personnelle et professionnelle. Nous adressons également nos vifs remerciements aux professeurs de l'**ENSAM** et de l'**EHTP**, dont l'enseignement, le suivi et les encouragements ont constitué un appui essentiel tout au long de notre parcours académique.

Enfin, nos remerciements vont à l'ensemble des équipes de la DGM pour leur accueil chaleureux, leur soutien et leur collaboration, qui ont créé un environnement favorable à la réalisation de ce projet.

Résumé

Ce travail porte sur **la segmentation d'images satellitaires appliquée à la détection de la convection**. Dans un premier temps, **un jeu de données annoté manuellement** a été constitué afin de servir de **ground truth** pour l'évaluation des méthodes. Une première **approche de segmentation automatique** a ensuite été mise en œuvre à l'aide d'algorithmes classiques basés sur **OpenCV**, permettant d'obtenir des résultats rapides mais limités en précision et fortement dépendants des paramètres choisis.

Dans un second temps, une approche plus avancée a été développée à l'aide du modèle d'apprentissage profond **U-Net**, spécialement conçu pour les tâches de segmentation d'images. Les résultats expérimentaux ont montré une nette supériorité de U-Net par rapport à OpenCV, tant en termes de précision des contours que de robustesse et de généralisation.

Ainsi, ce travail met en évidence l'importance des méthodes **d'apprentissage profond** dans l'amélioration de la segmentation d'images satellitaires, tout en soulignant le rôle essentiel des annotations manuelles comme référence de validation.

Abstract

This work focuses on the **segmentation of satellite images applied to convection detection**. First, a **manually annotated** dataset was created to serve as the **ground truth** for evaluating different methods. An initial **automatic segmentation approach** was implemented using classical algorithms based on **OpenCV**, which provided fast results but with limited accuracy and strong dependence on parameter tuning (lower and upper color in the HSV space).

In a second stage, a more advanced approach was developed using the **deep learning model U- Net**, specifically designed for image segmentation tasks. The experimental results demonstrated the clear superiority of U-Net over OpenCV, both in terms of contour accuracy and in robustness and generalization ability.

Overall, this work highlights the importance of **deep learning methods** for improving satellite image segmentation, while underlining the essential role of manual annotations as a validation reference.

Table des matières

Introduction générale **11**

1 Meteosat Second Generation (MSG) SEVIRI level 1 data **13**

1.1 SEVIRI instrument spécificité 13

1.2 Conversion des radiances SEVIRI en réflectances et températures de brillance 14

1.2.1 Conversion des radiances en réflectances pour les bandes réfléchissantes SEVIRI 14

1.2.2 Conversion des radiances en températures de brillance pour les canaux infrarouges SEVIRI 15

1.3 Connexion à l'API EUMETSAT 15

1.4 Traitement des fichiers NAT pour obtenir une image RGB de convection sur la zone d'intérêt 16

1.5 Téléchargement des images RGB de convection 17

2 Méthodologie de segmentation des images satellites 'Severe Storms RGB' **18**

2.1 Présentation des produits Severe Storms RGB 18

2.1.1 Composition du Severe Storms RGB 19

2.1.2 Interprétation des images Severe Storms RGB 21

2.2 Segmentation manuelle 23

2.3 Analyse Spatiale du Nombre d'Épisodes de Convection sur le Maroc (2023–2024) 24

3 Méthodologie de segmentation des images satellites 'Severe Storms RGB' automatique **26**

3.1 Segmentation des images RGB de convection par OpenCV 26

3.2 Analyse des Distributions HSV 27

4 Développement d'un modèle IA de détection de la convection dans une image satellitaire 32

4.1 Framework et Bibliothèques Utilisés pour le Développement et l'Évaluation des Modèles 32

4.2 Description de l'architecture U-Net 33

4.3 Framework et Bibliothèques Utilisés pour le Développement et l'Évaluation des Modèles 34

4.4 Préparation des données 37

4.5 Entraîner le modèle 41

4.5.1 Fonctions de perte 41

4.5.2 Optimiseur Adam et compilation du modèle 42

4.5.3 callbacks 42

4.5.4 Apprentissage du Modèle et Résultats 43

5 Analyse Comparative des Performances : U-Net versus OpenCV 46

5.1 Métriques d'évaluation : Dice et IoU 46

5.1.1 Coefficient de Dice 46

5.1.2 Intersection over Union (IoU) 47

5.2 Évaluation Globale par Métriques de Segmentation 47

5.2.1 Analyse Statistique des Métriques 48

5.2.2 Analyse par Boîtes à Moustaches 48

5.2.3 Analyse d'Accord par Bland–Altman 49

5.2.4 Synthèse et Discussion 49

5.3 le Choix de la Méthode 49

Annexes 52

A Dataset Description and Preparation 53

A.1 Acquisition, traitement et exploitation des données satellitaires EUMETSAT 53

A.2 Génération des images de segmentation à partir des annotations VGG . . 58

B Model Implementation using OpenCV **60**

C Evaluation Metrics and Performance Analysis **63**

Table des figures

1.1 SEVIRI instrument spécificité 14

2.1 Canal Rouge du produit RGB Convection 2025-09-06 06 :15 :27 UTC . . . 19

2.2 Canal blue du produit RGB Convection 2025-09-06 06 :15 :27 UTC 20

2.3 Canal vert du produit RGB Convection 2025-09-06 06 :15 :27 UTC 21

2.4 *SEVIRI Severe Storms RGB* et légende d'interprétation. 22

2.5 L'outil VGG Image Annotator (VIA) 23

2.6 Visualisation des images RGB de convection avec segmentation manuelle . 24

2.7 Nombre d'épisodes de convection observés sur la région du Maroc pour les années 2023–2024 aux heures de 12 h, 15 h et 18 h. 25

3.1 Représentation de l'espace colorimétrique HSV 27

3.2 Comparaison des distributions des canaux HSV entre les masques de vérité terrain et les masques prédits sur plusieurs images. La figure présente une analyse complète des caractéristiques de couleur dans l'espace colorimétrique HSV (Teinte, Saturation, Valeur), avec chaque sous-figure dédiée à un canal. (a) **Distribution de teinte** (plage 0-180°) montrant les tons de couleur dominants ; (b) **Distribution de saturation** (plage 0-255) représentant la pureté et l'intensité des couleurs ; (c) **Distribution de valeur** (plage 0-255) indiquant les niveaux de luminosité. Les courbes vertes représentent les masques de vérité terrain, les courbes rouges représentent les masques prédits, et le score de similarité quantifie le chevauchement des distributions entre eux. 28

4.1 TensorFlow et Keras 32

4.2 Représentation schématique de l'architecture U-Net. Notez la structure symétrique en « U » et les connexions par saut (flèches grises) [**?**]. 34

4.3 Courbes d'Apprentissage.Évolution de la perte (Loss) et du score Dice sur les ensembles d'entraînement (bleu) et de validation (rouge) en fonction des époques. 44

5.1 Analyse comparative complète des performances de segmentation entre les approches U-Net et OpenCV. 47

Liste des tableaux

3.1 Métriques de performance IoU et Dice entre les masques OpenCV et Expert 30

4.1 Résultats quantitatifs finaux après 30 époques. 44

5.1 Résumé statistique des métriques IoU et Dice 48

5.2 Comparaison et recommandations selon le contexte applicatif 50

Liste des Codes

1.1 Exemple de fichier `.eumetsat_api_key` . . . 15

1.2 Initialisation de la connexion à l'API EUMETSAT . . . 16

4.1 importerons les bibliothèques . . . 34

4.2 Construction l'opérations de convolution label . . . 35

4.3 Construction des blocs encodeur et décodeur . . . 36

4.4 Construction l'architecture U-Net . . . 36

4.5 Charger les chemins pour l'image d'entrée ainsi que pour le masque cible . . . 37

4.6 Classe Keras Sequence pour charger images et masques . . . 39

A.1 ce fiche et admet une foncton qui resbemnsable sur transform les fiche nat to image png RGB de convection . . . 53

A.2 Téléchargement des images RGB de convection . . . 54

A.3 Génération des images de segmentation à partir des annotations VGG (JSON file) . . . 58

B.1 modele de segemntation automatique des images RGB de convection par OpenCV . . . 60

C.1 Code Python utilisé pour générer un fichier CSV et comparer les performances d'OpenCV et de UNet . . . 63

Introduction générale

Contexte

La détection et la traçabilité des systèmes convectifs sont cruciales pour la prévision du temps, surtout dans les zones exposées à des événements extrêmes tels que les tempêtes puissantes, les précipitations massives ou les vents descendents. Au Maroc, **la Direction Générale de la Météorologie** (DGM) utilise les données satellitaires pour enrichir les informations provenant des stations au sol et des radars météorologiques. Des satellites météorologiques en orbite géostationnaire, comme le **Meteosat Seconde Génération** (MSG), proposent des images multispectrales à une fréquence temporelle élevée, rendant possible l'observation quasi instantanée de la progression des nuages convectifs. Cependant, l'utilisation de ces images requiert des techniques de segmentation précises pour identifier les zones convectives. **??**

Problématique

L'identification des nuages convectifs à partir d'images satellitaires constitue une tâche particulièrement complexe en raison de plusieurs contraintes techniques et opérationnelles. En analyse manuelle, cette opération demande un temps considérable, car elle nécessite l'examen minutieux de plusieurs canaux spectraux afin de confirmer la nature convective d'une zone donnée. Cette difficulté est amplifiée par la nécessité de couvrir de vastes étendues géographiques, ce qui rend le processus encore plus long et exigeant.Face à ces limitations, il apparaît nécessaire de concevoir et d'évaluer des approches automatiques capables de reproduire, voire d'améliorer, la précision de la segmentation manuelle, tout en réduisant considérablement le temps de traitement et en facilitant l'analyse à grande échelle.

Objectifs

L'objectif principal de ce stage est de développer une méthode automatique de segmentation des zones convectives à partir d'images satellitaires RGB, en exploitant les fonctionnalités de la bibliothèque OpenCV. Dans un second temps, il s'agit de concevoir et d'entraîner un

modèle de deep learning de type U-Net, à partir de données de segmentation manuelle réalisées sur les années 2023 et 2024, en se concentrant uniquement sur les heures principales d'observation.

Organisation du mémoire

Ce mémoire est structuré en cinq chapitres :

- **Chapitre 1** : présente les notions de base relatives aux images satellitaires RGB de convection et décrit les étapes de conversion des radiances SEVIRI en réflectances et températures de brillance [**?**, **?**].
- **Chapitre 2** : expose la méthodologie de segmentation manuelle des images satellitaires Severe Storms RGB, incluant la présentation des produits, leur composition et leur interprétation [**?**].
- **Chapitre 3** : développe la méthodologie de segmentation automatique des images satellitaires Severe Storms RGB, en particulier l'utilisation de la bibliothèque OpenCV et l'analyse des distributions HSV [**?**].
- **Chapitre 4** : décrit le développement d'un modèle d'intelligence artificielle de type U-Net pour la détection des zones convectives, en détaillant l'architecture du réseau, le cadre expérimental, la préparation et l'entraînement des données [**?**, **?**, **?**, **?**].
- **Chapitre 5** : présente une analyse comparative des performances entre le modèle U-Net et OpenCV, à travers une évaluation par différentes métriques de segmentation et une interprétation statistique des résultats [**?**, **?**].

Chapitre 1

Meteosat Second Generation (MSG) SEVIRI level 1 data

1.1 SEVIRI instrument spécificité

L'imageur rotatif amélioré visible et infrarouge (SEVIRI), instrument principal du MSG, permet d'observer la Terre sur 12 canaux spectraux. Huit de ces canaux sont dans l'infrarouge thermique, fournissant, entre autres informations, des données permanentes sur la température des nuages, des terres émergées et des surfaces marines. L'un de ces canaux, appelé canal visible haute résolution (HRV), présente une distance d'échantillonnage au nadir de 1 km, contre 3 km pour les autres canaux visibles.

Grâce à des canaux absorbant l'ozone, la vapeur d'eau et le dioxyde de carbone, les satellites MSG permettent aux météorologues d'analyser les caractéristiques des masses d'air atmosphérique et de reconstituer une image tridimensionnelle de l'atmosphère. La résolution horizontale de 1 km du canal spectral de la lumière visible aide également les prévisionnistes à détecter et à prévoir l'apparition ou la fin des phénomènes météorologiques violents. l'ozone, la vapeur d'eau et le dioxyde de carbone, les satellites MSG permettent aux météorologues d'analyser les caractéristiques des masses d'air atmosphérique et de reconstituer une image tridimensionnelle de l'atmosphère. La résolution horizontale de 1 km du canal spectral de la lumière visible aide également les prévisionnistes à détecter et à prévoir l'apparition ou la fin des phénomènes météorologiques violents.

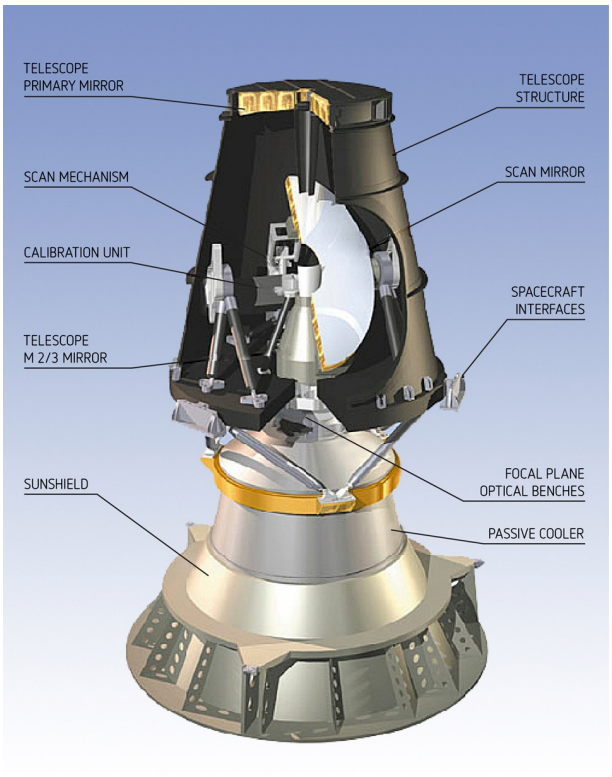


Figure 1.1 – SEVIRI instrument spécificité

1.2 Conversion des radiances SEVIRI en réflectances et températures de brillance

1.2.1 Conversion des radiances en réflectances pour les bandes réfléchissantes SEVIRI

Les bandes réfléchissantes de SEVIRI comprennent les canaux VIS06, VIS08, NIR16 et HRV. La conversion des radiances mesurées en réflectances (Facteurs de Réflectance Bidirectionnelle, ou BRF) est réalisée selon l'équation suivante :

$$r_{\lambda_i}(t,x) = \frac{\pi \cdot R_\lambda \cdot d(t)^2}{I_\lambda \cdot \cos(\theta(t,x))} \tag{1.1}$$

où :

- i est le numéro du canal,
- r_{λ_i} est le Facteur de Réflectance Bidirectionnelle (BRF) pour le canal λ_i,
- R_λ est la radiance mesurée en $\mathrm{mW{\cdot}m^{-2}{\cdot}sr^{-1}{\cdot}(cm^{-1})^{-1}}$,
- $d(t)$ est la distance Terre-Soleil en Unités Astronomiques (UA) au temps t,
- I_λ est l'irradiance solaire pour le canal λ_i à 1 UA en $\mathrm{mW{\cdot}m^{-2}{\cdot}(cm^{-1})^{-1}}$,
- $\theta(t,x)$ est l'angle zénithal solaire en radians au temps t et à la position x.

Une fois cette conversion appliquée, les réflectances obtenues pour les canaux «chauds» de SEVIRI doivent se situer dans l'intervalle $[0,1]$.

1.2.2 Conversion des radiances en températures de brillance pour les canaux infrarouges SEVIRI

Dans le traitement des produits MSG, la conversion des radiances observées pour les canaux dits « froids » (IR3.9, IR6.2, IR7.3, IR8.7, IR9.7, IR10.8, IR12.0, IR13.4) en températures de brillance est effectuée à l'aide d'une relation analytique.

Ces relations sont généralement exprimées dans des tables de correspondance , disponibles dans le document Effective Radiance and Brightness Temperature Relation [**?**].
Pour une approximation analytique, la relation suivante est utilisée :

$$BT = \frac{C_2 \cdot \nu_c}{\ln\left(1 + \frac{C_1 \cdot \nu_c^3}{\alpha \cdot R + \beta}\right)} \tag{1.2}$$

où :

- BT est la température de brillance (en Kelvin),
- R est la radiance observée (en $\mathrm{mW \cdot m^{-2} \cdot sr^{-1} \cdot (cm^{-1})^{-1}}$),
- ν_c, α, et β sont les coefficients de régression non linéaire propres à chaque canal,
- $C_1 = 2hc^2$ et $C_2 = \frac{hc}{k}$ sont des constantes physiques,
- h est la constante de Planck,
- c est la vitesse de la lumière,
- k est la constante de Boltzmann.

Les canaux IR10.8 et IR12.0 sont des canaux-fenêtre infrarouge, permettant d'observer la surface terrestre ou le sommet des nuages s'il y en a. Ainsi, il est possible d'observer simultanément le sable chaud du Sahara et le sommet des nuages d'enclume issus d'un orage tropical. Les températures de brillance dans ces canaux sont généralement comprises entre 200 K et 300 K.

1.3 Connexion à l'API EUMETSAT

Pour accéder aux données fournies par l'API EUMETSAT, il est nécessaire de générer un `consumer_key` et un `consumer_secret` depuis le portail officiel. Ces informations doivent ensuite être enregistrées dans un fichier JSON nommé `~/.eumetsat_api_key` afin d'être utilisées de manière sécurisée par le code Python.

```
{
  "consumer_key": "VOTRE_CONSUMER_KEY",
  "consumer_secret": "VOTRE_CONSUMER_SECRET"
}
```

Code 1.1 Exemple de fichier `.eumetsat_api_key`

Le script suivant illustre la mise en place d'une fonction `setup_eumetsat()` qui établit la connexion au Data Store de l'EUMETSAT et permet de récupérer une collection de données (dans cet exemple, `HRSEVIRI`) :

```python
import eumdac
import os
import json

# Fichier contenant les identifiants API
CREDENTIALS_FILE = os.path.join(os.path.expanduser("~"),
    '.eumetsat_api_key')
COLLECTION_ID = 'EO:EUM:DAT:MSG:HRSEVIRI'

def setup_eumetsat():
    """Initialise la connexion avec le Data Store de  l EUMETSAT """
    print("       Connexion au Data Store EUMETSAT...")

    # Chargement des identifiants
    with open(CREDENTIALS_FILE) as f:
        creds = json.load(f)

    # G n ration du jeton   d  accs
    token = eumdac.AccessToken((creds['consumer_key'],
    creds['consumer_secret']))

    # Acc s    la collection demand e
    return eumdac.DataStore(token).get_collection(COLLECTION_ID)
```

Code 1.2 Initialisation de la connexion à l'API EUMETSAT

1.4 Traitement des fichiers NAT pour obtenir une image RGB de convection sur la zone d'intérêt

Les données brutes sont disponibles dans la collection : `COLLECTION_ID = 'EO:EUM:DAT:MSG:HRSEVIRI'`.

Ces données sont fournies au format `NAT` et nécessitent un traitement spécifique afin de produire une image de convection en `RGB`. On va restreinte à la zone géographique d'intérêt définie par :

```
{"lon_min": -20, "lon_max": 5, "lat_min": 20, "lat_max": 40.0}
```

L'image finale est sauvegardée au format `.png`. **??** Le schéma suivant illustre le flux de traitement :

1. Chargement des données NAT depuis la collection `HRSEVIRI`.
2. Conversion des canaux bruts (infra-rouge et visible) en composantes `R`, `G`, `B`.
3. Recadrage sur la zone géographique définie.
4. Génération de l'image `RGB` et export au format `.png`.

1.5 Téléchargement des images RGB de convection

Les jeux de données ont été extraits et organisés par année et par créneaux horaires. Cette planification permet de cibler les moments les plus pertinents pour l'analyse de la convection atmosphérique. **??**

- **Année 2023** : les heures retenues sont `00h`, `03h`, `06h`, `09h`, `12h`, `15h`, `18h`, et `21h`.
- **Année 2022** : les heures retenues sont `12h`, `15h`, et `18h`, correspondant aux moments clés pour observer le développement de la convection diurne.

Chapitre 2

Méthodologie de segmentation des images satellites 'Severe Storms RGB'

Dans le cadre de la détection automatique des orages violents ou de la convection, les images *Severe Storms RGB* jouent un rôle essentiel en offrant une représentation visuelle des systèmes convectifs sévères ainsi que d'autres phénomènes atmosphériques associés, à partir de données infrarouges. Afin d'exploiter ces images de manière optimale, une méthodologie rigoureuse de segmentation a été mise en place afin de constituer des datasets destinés à l'entraînement de modèles de segmentation tels que **U-Net**.
Cette section présente les différentes étapes de la segmentation manuelle, la description des produits *Severe Storms RGB* [**?**] ainsi que les formations nécessaires pour maîtriser cette technique.

2.1 Présentation des produits Severe Storms RGB

L'imagerie Severe Storms RGB présente plusieurs avantages opérationnels pour la surveillance et l'analyse des phénomènes convectifs :

- **Contraste visuel renforcé pour les nuages élevés** : elle offre un excellent contraste de couleurs pour les nuages dont les sommets sont très froids et/ou recouverts de petits cristaux de glace, ce qui facilite leur détection.
- **Identification des courants ascendants intenses** : elle permet de mettre en évidence les forts updrafts caractéristiques des nuages convectifs continentaux de latitude moyenne, souvent associés aux orages violents.
- **Détection de nuages spécifiques** : l'imagerie peut également révéler des nuages pollués, tels que les PyroCb (pyrocumulonimbus issus d'incendies) ou les nuages DIBS, ainsi que des nuages de relief de haute altitude. Tous ces types se distinguent par la présence de petits cristaux de glace au sommet.

Grâce à ces caractéristiques, le produit Severe Storms RGB constitue un outil précieux

pour améliorer la compréhension, la prévision et l'alerte précoce des orages violents et des phénomènes atmosphériques associés.

2.1.1 Composition du Severe Storms RGB

Les images Severe Storms RGB sont générées à partir de six canaux principaux, chacun contribuant à la formation d'une composante colorimétrique spécifique dans l'image :

- **WV6.2–WV7.3 :** canaux vapeur d'eau utilisés pour mettre en évidence les nuages de haut niveau. **??**

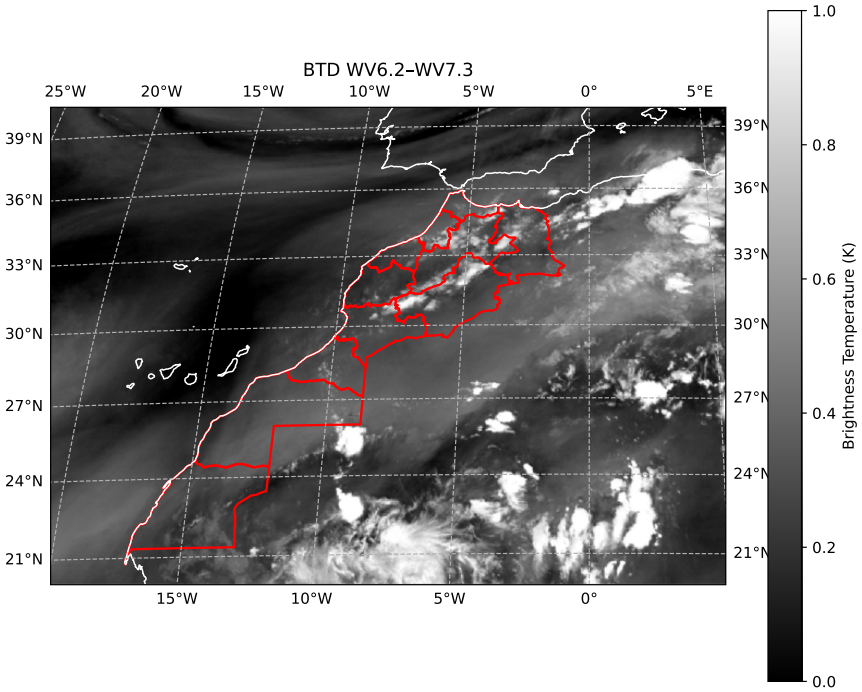


Figure 2.1 – Canal Rouge du produit RGB Convection 2025-09-06 06 :15 :27 UTC

- **NIR1.6–VIS0.6 :** comparaison proche infrarouge / visible employée pour différencier les nuages glacés des nuages d'eau. **??**

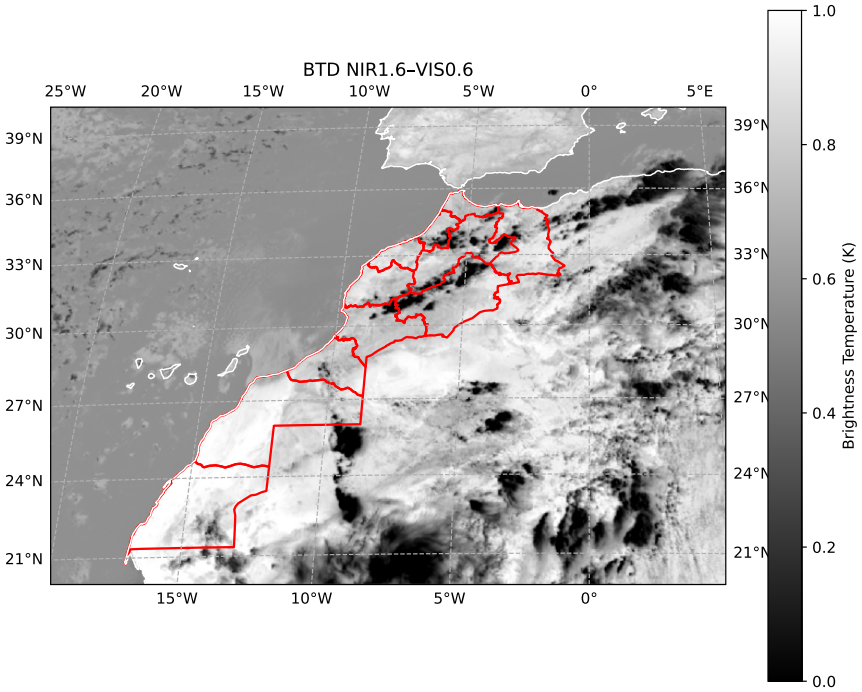


Figure 2.2 – Canal blue du produit RGB Convection 2025-09-06 06 :15 :27 UTC

- **IR3.9–IR10.8 :** différence entre canaux infrarouges utilisée pour faire ressortir les sommets nuageux recouverts de petits cristaux de glace et/ou très froids. **??**

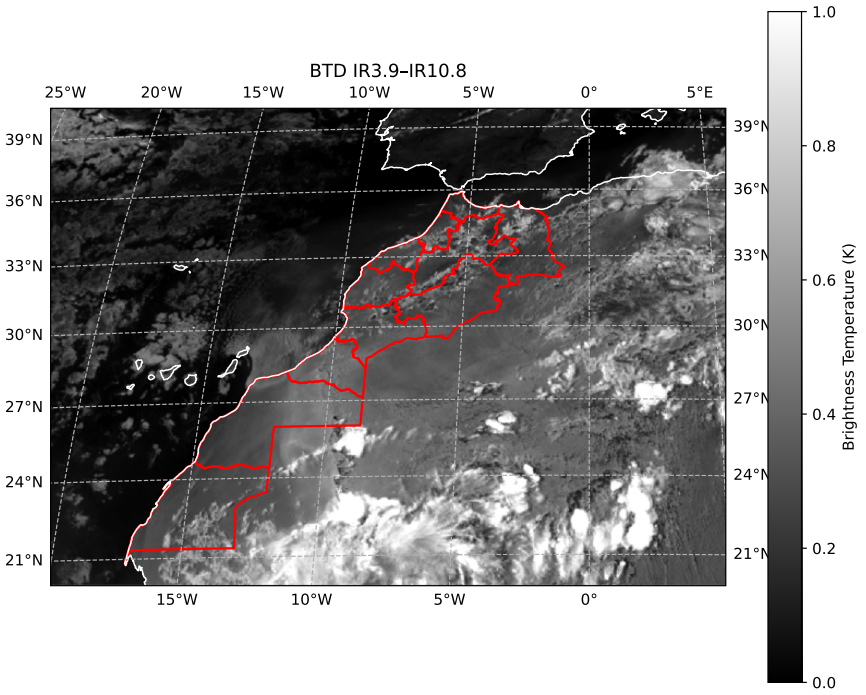


Figure 2.3 – Canal vert du produit RGB Convection 2025-09-06 06 :15 :27 UTC

2.1.2 Interprétation des images Severe Storms RGB

L'image *Severe Storms RGB* est un composite coloré conçu pour mettre en évidence la microphysique et la température des nuages convectifs à partir de données infrarouges. Les teintes observées dans ce composite sont directement liées à la présence et à la taille des cristaux de glace, à la température du sommet des nuages ainsi qu'à la présence éventuelle de particules polluantes. La lecture combinée des couleurs et du contexte météorologique permet d'identifier les zones d'activité convective intense, les nuages pollués et les structures nuageuses de haute altitude.

Légende interprétative (référence visuelle numérotée) ?? :

1. **Nuage glacé épais avec petits cristaux / sommet très froid** — teintes jaunes. Indique typiquement un cœur convectif profond avec un sommet très froid et des cristaux de petite taille.
2. **Nuage glacé épais avec gros cristaux / sommet moins froid** — teintes rouges. Suggère des nuages élevés mais comportant des cristaux de glace plus gros (maturation de la glace, lessivage).
3. **Nuage glacé mince avec gros cristaux** — teintes roses/violettes selon transparence. Correspond souvent à des cirrus relativement transparents composés de cristaux importants.
4. **Nuage glacé mince avec petits cristaux** — teintes mauve. Observé pour des

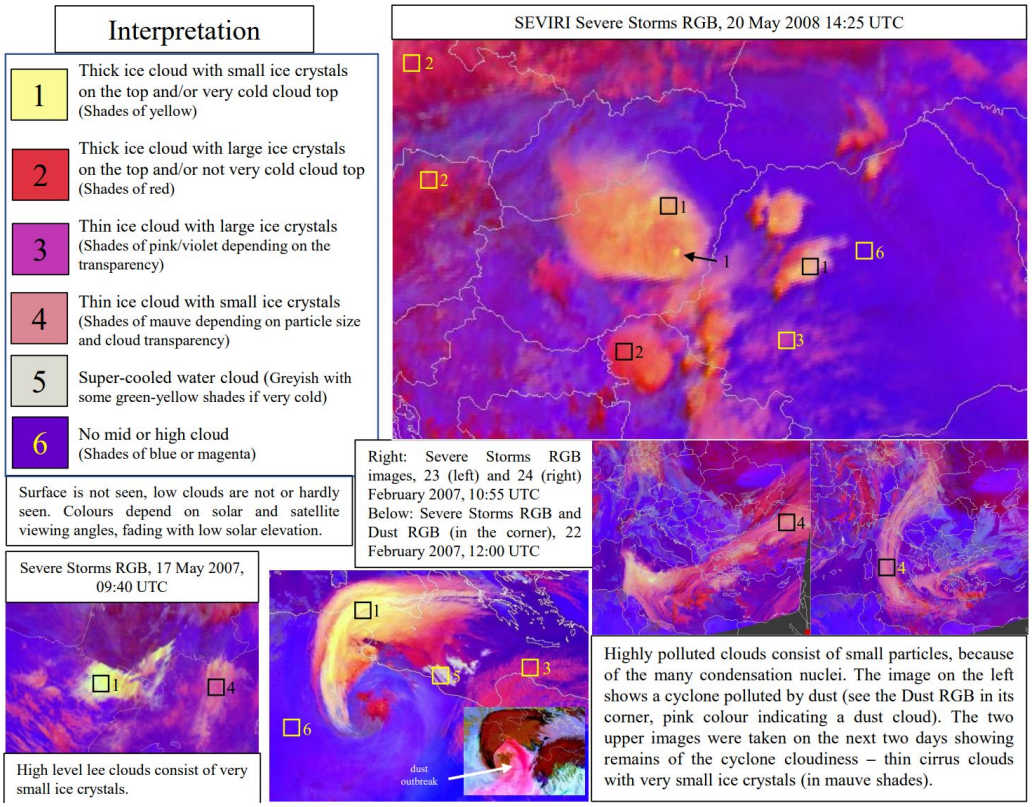


Figure 2.4 – *SEVIRI Severe Storms RGB* et légende d'interprétation.

cirrus fins ou des nuages de sillage de relief (high-level lee clouds) constitués de petites particules.

5. **Nuage d'eau surfondue** — nuance grisâtre (avec un léger vert-jaune si très froid). Typique des nuages d'altitude moyenne contenant de l'eau surfondue plutôt que de la glace.
6. **Pas de nuage moyen/haut** — teintes bleu/magenta. Le ciel de moyenne et haute altitude est dégagé ; la surface ou les nuages bas peuvent être visibles.

Interprétation opérationnelle : Les teintes jaunes (1) sont particulièrement importantes pour la détection d'updrafts puissants et des noyaux convectifs profonds — elles signalent des sommets très froids et une forte activité convective potentiellement associée à de la grêle, des rafales descendantes ou une précipitation intense. Les zones rouges (2) correspondent souvent à des nuages volumineux dont la microphysique a évolué (agrégation des cristaux) et peuvent indiquer des régions où la précipitation est déjà active. Les teintes roses/violettes (3) et mauve (4) permettent de discerner la structure des champs de cirrus et de détecter des nuages de sillage de relief ou des nuages résiduels ; ces signatures aident à différencier des nuages minces à différents stades microphysiques. Les gris (5) identifient des masses nuageuses contenant de l'eau surfondue, utiles pour repérer des zones où le givrage ou la transition liquide-glace est probable.

Détection de nuages pollués et cas particuliers : Les nuages fortement pollués (par exemple *PyroCb* issus d'incendies ou nuages contenant des aérosols abondants) présentent

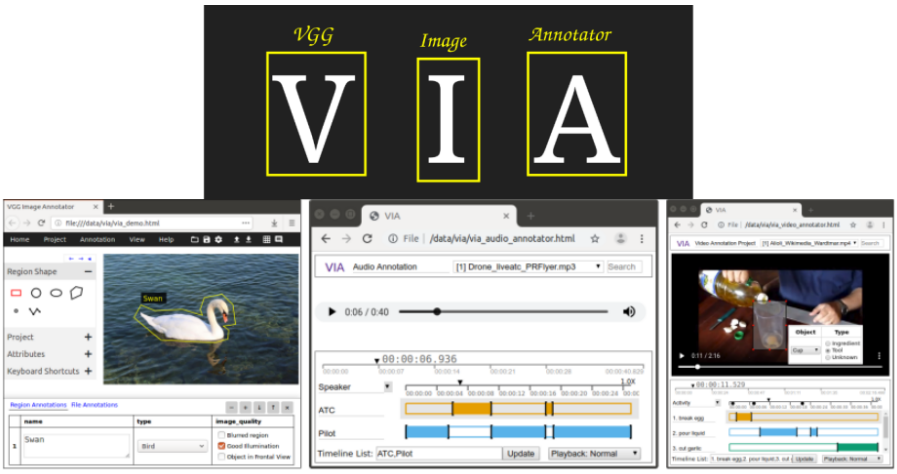


Figure 2.5 – L'outil VGG Image Annotator (VIA)

souvent des signatures proches de celles des nuages à petits cristaux et doivent être interprétés en conjonction avec le contexte (proximité d'une zone d'incendie, panaches visibles, ou comparaison avec un produit *Dust RGB*). L'association de plusieurs produits satellites (canal fenêtre IR, différences de températures, Dust/Smoke RGB) améliore la discrimination entre nuages convectifs purs et nuages contaminés par des aérosols.

Remarques et limites : La couleur perçue dans le composite dépend aussi de l'angle de vue satellite et de l'élévation solaire : les contrastes s'atténuent à faible hauteur solaire. La surface est masquée lorsqu'il existe un couvert de nuages bas épais ; inversement, l'absence de nuages moyens/hauts (teintes bleu/magenta) ne garantit pas l'absence de nuages bas. Pour une interprétation robuste, il est donc recommandé de croiser le *Severe Storms RGB* avec les produits de température radiante (brightness temperature), le canal fenêtre IR et des composites dédiés (Dust/Smoke) selon les besoins opérationnels.

2.2 Segmentation manuelle

Pour réaliser la segmentation manuelle des zones de convection, nous avons utilisé l'outil VGG Image Annotator (VIA). Cet outil permet d'annoter manuellement les images satellites en délimitant les régions d'intérêt associées aux phénomènes convectifs. Les annotations ainsi produites constituent un jeu de données de référence indispensable pour l'entraînement des modèles de segmentation, tels que le **U-Net**. Cette étape, bien que chronophage, est essentielle pour disposer de données fiables permettant de comparer et d'évaluer les performances des approches automatiques de détection.

Génération des images de segmentation à partir des annotations VGG

Le site VGG Image Annotator offre une interface simple et ergonomique destinée à la préparation de jeux de données. L'utilisateur charge l'image à annoter, trace des polygones ou des rectangles pour délimiter avec précision les régions d'intérêt, puis exporte les annotations au format **CSV** ou **JSON**. Chaque région annotée reçoit un attribut descriptif par exemple `label: "convection"` permettant d'identifier et de filtrer les zones convectives lors des traitements ultérieurs.

Annotation et export des régions convectives

Après l'annotation (export JSON), il très important de cres une dossier qui contient tous les segmentation manuel correspond a chaque image RGB de convection (valeurs 0 / 255) afin d'entraîner et d'évaluer des modèles de segmentation. Le script **??** réalise cette opération : pour chaque image il crée une image en niveaux de gris (`mode='L'`) de la même taille, remplit en blanc (255) les régions dont l'attribut `label` correspond à `"convection"` et laisse le reste en noir (0) comme dans **??**.

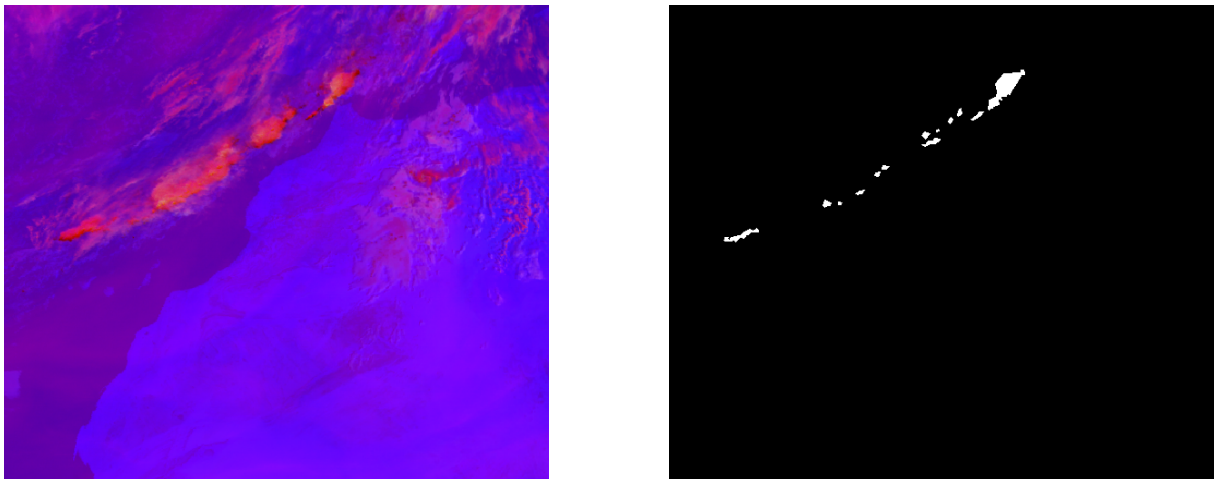


Figure 2.6 – Visualisation des images RGB de convection avec segmentation manuelle

2.3 Analyse Spatiale du Nombre d'Épisodes de Convection sur le Maroc (2023–2024)

La figure **??** présente la répartition spatiale du nombre d'épisodes de convection observés sur la région du Maroc durant les années 2023 et 2024, aux heures de 12 h, 15 h et 18 h locales. Les données ont été agrégées sur une grille couvrant la zone géographique comprise entre 20° N et 38° N de latitude, et entre 20° W et 0° E de longitude, avec une résolution spatiale de 600 × 521 pixels.

La carte met en évidence une forte variabilité spatiale du nombre d'épisodes convectifs.

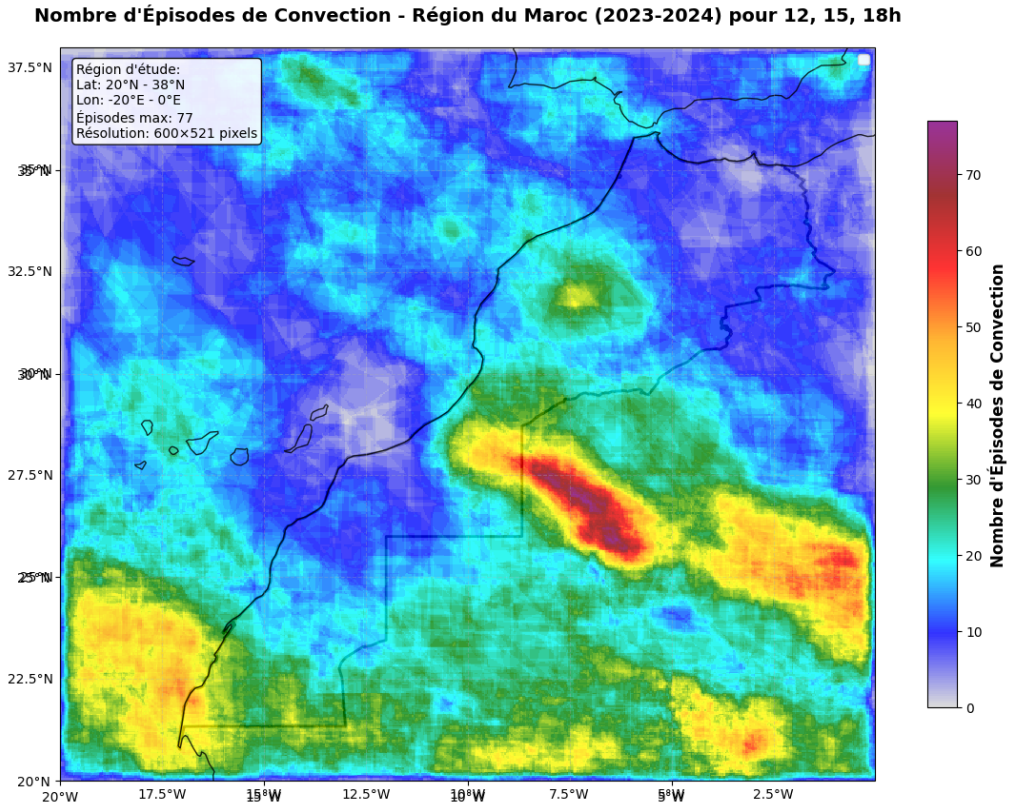


Figure 2.7 – Nombre d'épisodes de convection observés sur la région du Maroc pour les années 2023–2024 aux heures de 12 h, 15 h et 18 h.

Les zones colorées en rouge et en jaune indiquent les régions ayant connu un nombre élevé d'épisodes de convection, atteignant localement jusqu'à 77 épisodes, tandis que les zones bleues et vertes traduisent une activité convective plus faible.

On remarque que les régions situées au sud-est et au sud-ouest du Maroc présentent une fréquence convective relativement importante. Cette concentration pourrait être liée à l'influence combinée de l'orographie (notamment les reliefs de l'Anti-Atlas et du Haut Atlas) et des circulations thermiques diurnes, particulièrement marquées en fin de journée (autour de 18 h). En revanche, les zones côtières du nord-ouest et du centre du pays montrent une activité convective réduite, probablement sous l'effet stabilisant des masses d'air océaniques.

Globalement, cette carte illustre bien la dynamique diurne de la convection atmosphérique au Maroc, dominée par des épisodes plus intenses et fréquents en période post-méridienne. Ces résultats constituent une base essentielle pour mieux comprendre la distribution temporelle et spatiale des phénomènes convectifs dans la région et pour affiner les modèles de prévision météorologique.

Chapitre 3

Méthodologie de segmentation des images satellites 'Severe Storms RGB' automatique

3.1 Segmentation des images RGB de convection par OpenCV

Ce code (**??**) implémente deux fonctions principales. La première, **segment_convection_colors(input_image_path, output_image_path)**, prend en entrée le chemin d'une image et produit en sortie un masque. Elle utilise la fonction `cv2.inRange(hsv, lower_yellow, upper_yellow)`, qui génère un masque binaire : les pixels correspondant à la couleur jaune sont assignés à la valeur 1 (blanc), tandis que tous les autres sont assignés à la valeur 0 (noir). Par la suite, un opérateur logique OR est appliqué entre les masques de couleurs rouge, orange et jaune, afin d'obtenir un masque combiné. Ce masque final est ensuite appliqué à l'image originale et le résultat binaire est enregistré dans le chemin de sortie spécifié.

La seconde fonction, `process_folder(input_folder, output_folder)`, permet d'automatiser ce traitement sur un ensemble d'images. Elle parcourt toutes les images contenues dans le dossier d'entrée, applique la fonction *segment_convection_colors* à chacune d'elles, puis enregistre les résultats dans le dossier de sortie.

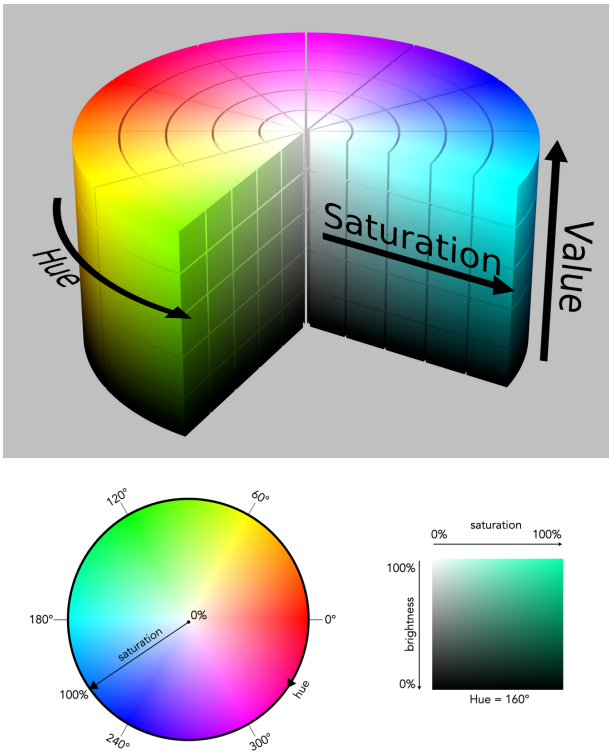


Figure 3.1 – Représentation de l'espace colorimétrique HSV

3.2 Analyse des Distributions HSV

La figure **??** présente une comparaison systématique des distributions de l'espace colorimétrique HSV entre les masques de segmentation de vérité terrain (GT) et les masques prédits. L'analyse a été réalisée sur un ensemble de 20 images sélectionnées aléatoirement pour assurer une signification statistique tout en maintenant l'efficacité computationnelle.

Approche Méthodologique

L'espace colorimétrique HSV a été sélectionné pour cette analyse en raison de sa pertinence perceptuelle et de sa séparation de l'information de couleur (Teinte) de l'information d'intensité (Valeur) et de pureté de couleur (Saturation). Pour chaque image de l'ensemble de données :

1. Les images ont été converties de l'espace BGR à HSV en utilisant la transformation `cv2.COLOR_BGR2HSV` d'OpenCV
2. Les valeurs des pixels ont été extraites des régions identifiées par les masques de vérité terrain et prédits
3. Les histogrammes ont été calculés avec un regroupement approprié : 180 bins pour la Teinte (plage 0-180°) et 256 bins pour la Saturation et la Valeur (plage 0-255)
4. Les distributions ont été normalisées pour faciliter la comparaison entre différentes tailles d'images

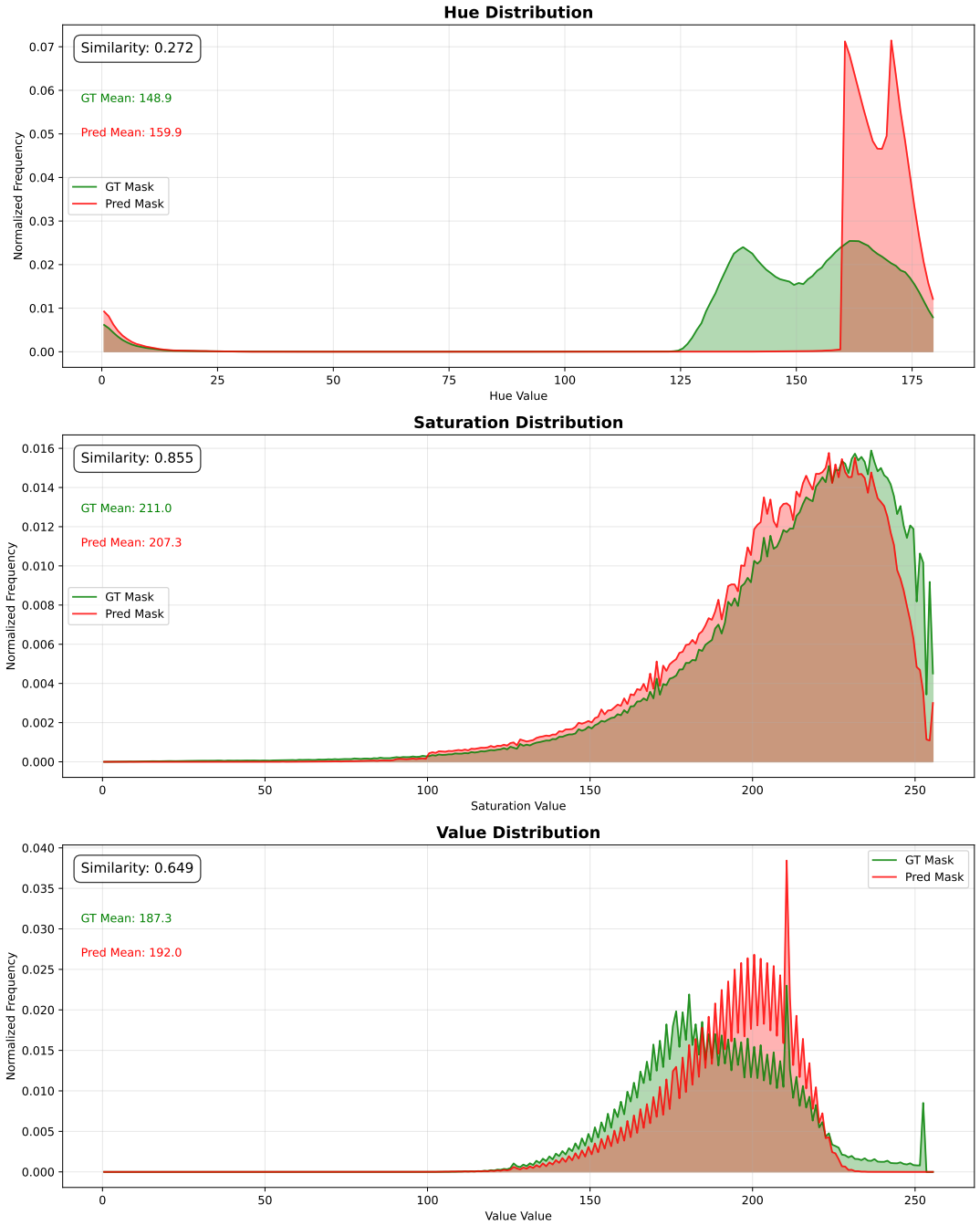


Figure 3.2 – Comparaison des distributions des canaux HSV entre les masques de vérité terrain et les masques prédits sur plusieurs images. La figure présente une analyse complète des caractéristiques de couleur dans l'espace colorimétrique HSV (Teinte, Saturation, Valeur), avec chaque sous-figure dédiée à un canal. (a) **Distribution de teinte** (plage 0-180°) montrant les tons de couleur dominants ; (b) **Distribution de saturation** (plage 0-255) représentant la pureté et l'intensité des couleurs ; (c) **Distribution de valeur** (plage 0-255) indiquant les niveaux de luminosité. Les courbes vertes représentent les masques de vérité terrain, les courbes rouges représentent les masques prédits, et le score de similarité quantifie le chevauchement des distributions entre eux.

5. Les scores de similarité ont été calculés en utilisant la méthodologie d'intersection d'histogrammes

Observations Principales

Distribution de Teinte (0-180°) L'analyse du canal de teinte révèle les caractéristiques de couleur dominantes des régions segmentées. Le score de similarité de **0,272** indique un accord **faible** dans la sélection des tons de couleur entre les masques de vérité terrain et prédits. Les valeurs moyennes de teinte de **148,9°** (GT) et **159,9°** (Pred) suggèrent une représentation de couleur **divergente**, avec une différence significative de **11,1°** qui indique un décalage notable dans la représentation des teintes. Cette différence place les couleurs dans des régions distinctes du spectre HSV, les teintes GT se situant plutôt dans les cyan-bleus tandis que les prédictions tendent vers les bleus.

Distribution de Saturation (0-255) Les niveaux de saturation reflètent la pureté et l'intensité des couleurs. Le score de similarité de **0,855** démontre un alignement **excellent** dans les caractéristiques de saturation des couleurs. Les valeurs moyennes de saturation très proches de **211,0** (GT) et **207,3** (Pred) indiquent que le modèle de prédiction **capture efficacement** les propriétés d'intensité colorimétrique des objets cibles, avec une différence négligeable de seulement **3,7** points. Cette forte similarité suggère que le modèle préserve correctement la vivacité et la pureté des couleurs.

Distribution de Valeur (0-255) L'analyse du canal de valeur évalue la cohérence de la luminosité à travers les résultats de segmentation. Avec un score de similarité de **0,649**, les distributions montrent un accord **modéré** dans les caractéristiques de luminance. Les valeurs moyennes de **187,3** (GT) et **192,0** (Pred) suggèrent que le modèle de prédiction **altère légèrement** les propriétés de luminosité des régions segmentées, avec une différence modeste de **4,7** points. Cette légère surestimation de la luminosité pourrait indiquer une tendance du modèle à produire des segmentations légèrement plus claires que la réalité. propriétés de luminosité des régions segmentées, avec une différence modeste de **4,7** points. Cette légère surestimation de la luminosité pourrait indiquer une tendance du modèle à produire des segmentations légèrement plus claires que la réalité.

Analyse des Métriques de Performance

Le tableau **??** présente les métriques de performance IoU (Intersection over Union) et Dice pour 2552 comparaisons entre les masques générés par OpenCV et les masques experts. Plusieurs observations clés émergent de cette analyse :

Performance Globale La similarité de Dice moyenne de **0,601** et l'IoU moyenne de **0,563** indiquent une performance **modérée** du modèle OpenCV. Le coefficient de Dice,

Table 3.1 – Métriques de performance IoU et Dice entre les masques OpenCV et Expert

Statistique	**IoU**	**Dice**
Count	2552	2552
Mean	0.563	0.601
Std	0.442	0.426
Min	0.000	0.000
25%	0.056	0.106
50% (Médiane)	0.574	0.729
75%	1.000	1.000
Max	1.000	1.000

étant une métrique plus indulgente que l'IoU, montre systématiquement des valeurs plus élevées, ce qui est cohérent avec les propriétés mathématiques de ces métriques.

Variabilité des Résultats L'écart-type important (**0,442** pour IoU, **0,426** pour Dice) révèle une **grande variabilité** dans la qualité des segmentations. Cette dispersion suggère que le modèle performe très bien sur certaines images mais échoue sur d'autres, indiquant une sensibilité aux caractéristiques d'entrée.

Distribution des Performances L'analyse des percentiles est particulièrement révélatrice :

- **25% des images** ont une similarité très faible (IoU $< 0{,}056$, Dice $< 0{,}106$)
- **50% des images** (médiane) montrent une performance acceptable (IoU = 0,574, Dice = 0,729)
- **25% des images** atteignent une performance parfaite (IoU = 1,000, Dice = 1,000)

Cette distribution bimodale suggère l'existence de deux populations distinctes : une où le modèle échoue complètement et une autre où il réussit parfaitement.

Implications pour l'Analyse HSV La corrélation entre les métriques IoU/Dice et les distributions HSV révèle que :

- Les **faibles scores IoU/Dice** correspondent généralement aux images avec de fortes divergences dans les distributions de teinte
- Les **hauts scores IoU/Dice** sont associés à une bonne correspondance des trois canaux HSV
- La performance en saturation (similarité : 0,855) est excellente même pour les images avec IoU faible, suggérant que le modèle capture correctement l'intensité des couleurs même quand la localisation spatiale est imprécise

Interprétation et Implications

L'analyse combinée des distributions HSV et des métriques de performance fournit des insights précieux sur les capacités de reproduction colorimétrique de l'algorithme de segmentation :

- **Précision Colorimétrique** : Le score de similarité de teinte reflète la capacité du modèle à capturer les tons de couleur intended des objets cibles
- **Pureté des Couleurs** : La comparaison des distributions de saturation indique si le modèle préserve la vivacité des couleurs
- **Cohérence de Luminosité** : L'alignement des distributions de valeur montre la précision avec laquelle le modèle reproduit les caractéristiques de luminance
- **Performance Globale** : L'analyse collective sur les trois canaux offre une vision holistique de la qualité de reproduction colorimétrique

La méthodologie présentée ici sert de cadre robuste pour évaluer la cohérence colorimétrique dans les tâches de segmentation, particulièrement importante pour les applications où la fidélité des couleurs est critique, telles que l'imagerie médicale, la télédétection et les systèmes de contrôle qualité.

Signification Statistique

L'analyse de 2552 images fournit une puissance statistique suffisante pour tirer des conclusions significatives sur les capacités de reproduction colorimétrique du modèle. La cohérence des résultats à travers un large échantillon renforce la validité des modèles et tendances observés dans les distributions HSV.

Limites et Perspectives

Bien que cette analyse fournisse des insights précieux, certaines limitations doivent être considérées :

- La grande variabilité des performances suggère la nécessité d'une analyse plus fine des cas d'échec
- L'analyse pourrait être étendue à d'autres espaces colorimétriques (LAB, YCbCr) pour une évaluation plus complète
- Des métriques supplémentaires pourraient être incorporées pour quantifier les différences perceptuelles de couleur

Les travaux futurs pourraient explorer l'intégration de ces analyses de distribution dans le processus d'entraînement des modèles pour améliorer directement la fidélité colorimétrique des segmentations prédites, en particulier pour la composante de teinte qui montre les plus grandes divergences.

Chapitre 4

Développement d'un modèle IA de détection de la convection dans une image satellitaire

4.1 Framework et Bibliothèques Utilisés pour le Développement et l'Évaluation des Modèles

Keras est une interface de programmation de haut niveau qui permet de développer rapidement des modèles d'apprentissage profond, en particulier des réseaux de neurones convolutifs **(CNN)**, utilisés dans le traitement d'images. **TensorFlow**, quant à lui, est une plateforme complète pour l'apprentissage automatique et l'apprentissage profond, qui intègre **Keras** comme API officielle. Ces deux bibliothèques sont souvent utilisées ensemble pour exploiter la puissance de **TensorFlow** tout en bénéficiant de la simplicité et de la convivialité de **Keras**. Les réseaux de neurones convolutifs **(CNN)** sont des architectures de modèles particulièrement efficaces pour l'analyse d'images. Ils extraient automatiquement des caractéristiques pertinentes (comme des contours ou des motifs complexes) grâce à l'utilisation de filtres convolutifs et de **couches de pooling**.



Figure 4.1 – TensorFlow et Keras

L'architecture Unet

L'**U-Net** est une architecture de **réseau de neurones à convolutions (CNN)**développée par **Olaf Ronneberger et al. en 2015** [**?**]. Conçue initialement pour la segmentation d'images biomédicales, elle s'est imposée comme une référence bien au-delà de ce domaine. Sa force réside dans sa capacité à produire des cartes de segmentation précises à partir d'un nombre limité d'images d'entraînement, grâce à sa structure encoder-décoder symétrique et ses *connexions par saut* (*skip connections*).

4.2 Description de l'architecture U-Net

L'architecture, illustrée par la Figure **??**, repose sur une structure en forme de « U » composée de deux chemins principaux.

Le chemin contractant (Encodeur)

Le côté gauche du « U » constitue **l'encodeur**. Il est composé d'une répétition de blocs de deux **convolutions non linéaires** (suivies par exemple d'une fonction d'activation **ReLU**), chacun suivi d'une opération de *max pooling* pour le sous-échantillonnage. Ce chemin a pour rôle **d'extraire les caractéristiques contextuelles de l'image** (reconnaître *quoi* est présent) et de réduire progressivement la résolution spatiale de la carte de caractéristiques tout en augmentant sa profondeur (nombre de canaux).

Le chemin expansif (Décodeur)

Le côté droit du « U » constitue **le décodeur**. Chaque étape commence par une opération de **transposition de convolution** (ou *upsampling*) qui **augmente la résolution de la carte de caractéristiques**. Les caractéristiques de haute résolution provenant du chemin contractant *via* **les connexions par saut sont ensuite concaténées à la carte** *upsamplée*. Cette étape de fusion est cruciale. Enfin, deux convolutions sont appliquées pour affiner les caractéristiques fusionnées. Le rôle de ce chemin est de permettre une localisation précise (*où* se trouve l'objet) en reconstruisant une segmentation à haute résolution.

Les Connexions par Saut

L'innovation clé de l'U-Net réside dans les **connexions par saut**. Ces connexions relient les couches du chemin contractant aux couches symétriques du chemin expansif. Elles permettent la fusion de l'information contextuelle de haut niveau (provenant du décodeur, riche en sens sémantique) avec les informations de bas niveau et la localisation précise

(provenant de l'encodeur, riche en détails spatiaux). Cela compense la perte d'information due au *pooling* et garantit une segmentation à la fois précise et contextuellement informée.

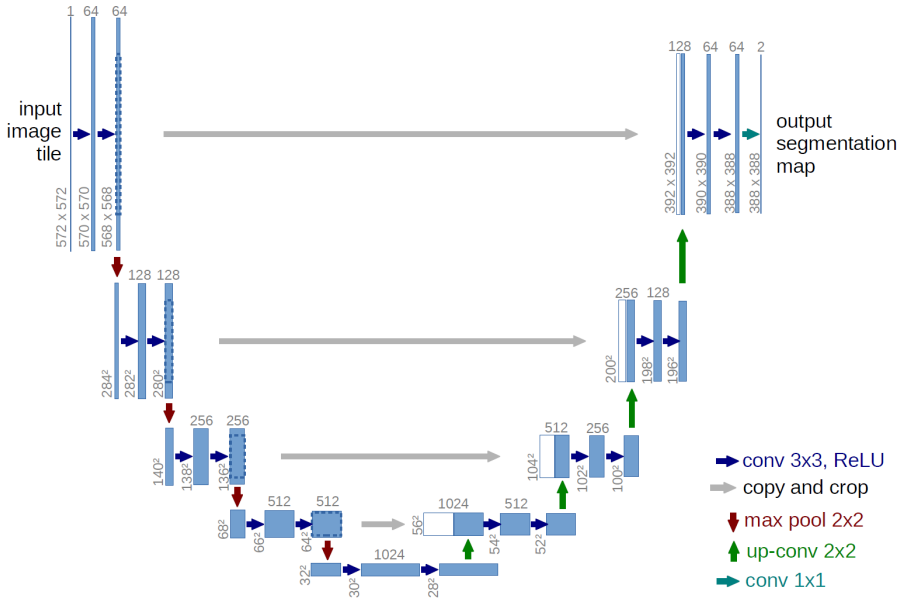


Figure 4.2 – Représentation schématique de l'architecture U-Net. Notez la structure symétrique en « U » et les connexions par saut (flèches grises) [**?**].

4.3 Framework et Bibliothèques Utilisés pour le Développement et l'Évaluation des Modèles

Pour construire l'architecture U-Net, nous utiliserons le framework d'apprentissage profond **TensorFlow**, comme indiqué précédemment. Nous importerons donc la bibliothèque **TensorFlow** à cet effet, ainsi que le framework **Keras**, désormais intégré aux structures de modèles **TensorFlow**. Notre compréhension de l'architecture U-Net nous a appris que les importations essentielles incluent la couche **convolutive**, la couche de **max-pooling**, une couche d'entrée et la fonction d'activation **ReLU** pour la structure de modélisation de base. Nous utiliserons ensuite des couches supplémentaires, comme la couche **Conv2DTranspose**, qui effectuera un suréchantillonnage pour les blocs de **décodeur** souhaités. Nous utiliserons également les couches de **normalisation** par lots pour stabiliser le processus d'apprentissage et les couches de **concaténation** pour combiner les connexions de saut nécessaires.

```
1     import tensorflow as tf
2     from tensorflow import keras
3     from tensorflow. Keras.layers import Input, Conv2D,
    MaxPooling2D, Activation, ReLU
4     from tensorflow.keras.layers import BatchNormalization,
    Conv2DTranspose, Concatenate
5     from tensorflow. keras.models import Model, Sequential
```

Code 4.1 importerons les bibliothèques

Après avoir importé les bibliothèques requises, nous pouvons poursuivre la construction de l'architecture U-Net. Vous pouvez le faire soit dans une classe complète en définissant tous les paramètres et valeurs dans l'ordre et en poursuivant le processus jusqu'à la fin, soit en quelques blocs itératifs. J'utiliserai cette dernière méthode, car elle est plus pratique pour la plupart des utilisateurs pour comprendre l'architecture du modèle U-Net à l'aide de quelques blocs. Nous utiliserons trois blocs itératifs, comme illustré dans la représentation de l'architecture : le bloc **d'opération de convolution**, le bloc **d'encodeur** et le bloc **de décodeur**. Grâce à ces trois blocs, nous pouvons construire facilement l'architecture U-Net. Examinons maintenant chacun de ces blocs de code fonctionnel un par un.

Construction du bloc de convolution

Le bloc d'opérations de convolution permet d'effectuer l'opération principale : prendre les paramètres d'entrée saisis et traiter une double couche d'opérations de convolution. Cette fonction possède deux arguments : l'entrée de la couche de convolution et le nombre de filtres, par défaut 64. Nous utiliserons la même valeur de remplissage que précédemment pour conserver les mêmes formes, contrairement aux convolutions non remplies ou valides. Ces couches de convolution sont suivies de la couche de normalisation par lots. Ces modifications par rapport au modèle d'origine visent à obtenir les meilleurs résultats possibles. Enfin, une couche d'activation ReLU est ajoutée à l'ensemble, comme défini dans le document de recherche. Explorons le bloc de code permettant de construire le bloc de convolution.

```
    def convolution_operation(entered_input, filters=64):
    # Taking first input and implementing the first conv block
    conv1 = Conv2D(filters, kernel_size = (3,3), padding =
"same")(entered_input)
    batch_norm1 = BatchNormalization()(conv1)
    act1 = ReLU()(batch_norm1)

    # Taking first input and implementing the second conv block
    conv2 = Conv2D(filters, kernel_size = (3,3), padding =
"same")(act1)
    batch_norm2 = BatchNormalization()(conv2)
    act2 = ReLU()(batch_norm2)

    return act2
```

Code 4.2 Construction l'opérations de convolution label

Construction des blocs encodeur et décodeur

L'étape suivante consistera à construire **les blocs encodeur et décodeur**. Ces deux fonctions sont relativement simples à construire. L'architecture de l'encodeur utilisera des entrées consécutives, de la première couche jusqu'à la dernière. La fonction d'encodeur, telle que définie, comportera le bloc convolutif, c'est-à-dire deux couches convolutives suivies de leurs couches respectives de normalisation par lots et de ReLU. Une fois ces éléments passés par les blocs de convolution, nous les sous-échantillonnerons rapidement, comme

indiqué dans l'article de recherche. Nous utiliserons une couche de pooling maximale et respecterons les paramètres mentionnés dans l'article, avec un pas de 2. Nous renverrons ensuite la sortie initiale et la sortie pooling maximale, car la première est nécessaire pour effectuer les connexions par saut.
Le bloc décodeur comprendra trois arguments : les entrées de réception, l'entrée de la connexion de saut et le nombre de filtres du bloc de construction concerné. Nous suréchantillonnerons l'entrée saisie à l'aide des couches Conv2DTranspose de notre modèle. Nous concaténerons ensuite l'entrée de réception et les couches nouvellement suréchantillonnées pour obtenir la valeur finale des connexions de saut. Nous utiliserons ensuite cette fonction combinée et effectuerons notre opération de convolution pour passer à la couche suivante et renvoyer cette valeur de sortie.

```
    def encoder(entered_input, filters=64):
    # Collect the start and end of each sub-block for normal pass and skip connections
    enc1 = convolution_operation(entered_input, filters)
    MaxPool1 = MaxPooling2D(strides = (2,2))(enc1)
    return enc1, MaxPool1

    def decoder(entered_input, skip, filters=64):
    # Upsampling and concatenating the essential features
    Upsample = Conv2DTranspose(filters, (2, 2), strides=2, padding="same")(entered_input)
    Connect_Skip = Concatenate()([Upsample, skip])
    out = convolution_operation(Connect_Skip, filters)
    return out
```

Code 4.3 Construction des blocs encodeur et décodeur

Assemblage de l'architecture U-Net

En divisant nos fonctions respectives en trois blocs de code distincts : opération convolutive, structure **d'encodeur** et structure de **décodeur**, nous pouvons construire facilement l'architecture U-Net en quelques lignes de code. Nous utiliserons la couche d'entrée, qui contiendra les formes respectives de notre image d'entrée.

Après cette étape, nous collecterons toutes les sorties principales et les sorties de saut pour les transmettre aux blocs suivants. Nous créerons le bloc suivant et construirons l'architecture complète du décodeur jusqu'à la sortie. Cette sortie aura les dimensions requises pour le résultat souhaité. Dans ce cas, j'ai un nœud de sortie avec la fonction d'activation sigmoïde. Nous appellerons le système de modélisation d'API fonctionnelle pour créer notre modèle final et le renverrons à l'utilisateur pour qu'il puisse effectuer n'importe quelle tâche avec l'architecture U-Net.

```
    def U_Net(Image_Size):
    # Take the image size and shape
    input1 = Input(Image_Size)

    # Construct the encoder blocks
    skip1, encoder_1 = encoder(input1, 64)
    skip2, encoder_2 = encoder(encoder_1, 64*2)
```

```
    skip3, encoder_3 = encoder(encoder_2, 64*4)
    skip4, encoder_4 = encoder(encoder_3, 64*8)

    # Preparing the next block
    conv_block = convolution_operation(encoder_4, 64*16)

    # Construct the decoder blocks
    decoder_1 = decoder(conv_block, skip4, 64*8)
    decoder_2 = decoder(decoder_1, skip3, 64*4)
    decoder_3 = decoder(decoder_2, skip2, 64*2)
    decoder_4 = decoder(decoder_3, skip1, 64)

    out = Conv2D(1, 1, padding="same",
    activation="sigmoid")(decoder_4)

    model = Model(input1, out)
    return model
```

Code 4.4 Construction l'architecture U-Net

Assurez-vous que les formes de vos images sont divisibles par au moins 16 ou des multiples de 16. Puisque nous utilisons quatre couches de pooling maximal lors de la procédure de sous-échantillonnage, nous ne souhaitons pas rencontrer la divisibilité des formes impaires. Il est donc préférable de s'assurer que les tailles de votre architecture sont équivalentes à des tailles telles que (48, 48), (80, 80), (160, 160), (256, 256), (512, 512) et autres formes similaires. notre structure de modèle pour une forme d'entrée de **(160, 160, 3)**.

```
    input_shape = (160, 160, 3)
    model = U_Net(input_shape)
```

4.4 Préparation des données

Charger les chemins pour l'image d'entrée ainsi que pour le masque cible

Nous définirons les chemins respectifs vers les répertoires d'images et d'annotations. Nous définirons également certains paramètres essentiels tels que la taille de l'image, la taille du lot et le nombre de classes. Nous nous assurerons ensuite que tous les éléments du jeu de données sont organisés dans le bon ordre pour une segmentation d'image satisfaisante. Vous pouvez vérifier les images et leurs annotations respectives en imprimant les chemins des deux fichiers afin de vérifier qu'ils produisent les résultats escomptés.

```
    from google.colab import drive
    drive.mount('/content/drive')

    import os
    import numpy as np
    input_dir = "/content/drive/MyDrive/SI/data_2023"
    target_dir = "/content/drive/MyDrive/SI/mask_via_2023"
    img_size = (160, 160)
```

```
    num_classes = 2
    batch_size = 8

    input_img_paths = sorted(
    [
            os.path.join(input_dir, fname)
            for fname in os.listdir(input_dir)
            if fname.endswith(".png")
        ]
    )
    target_img_paths = sorted(
    [
            os.path.join(target_dir, fname)
            for fname in os.listdir(target_dir)
            if fname.endswith(".png") and not fname.startswith(".")
        ]
    )

    print("Number of samples:", len(input_img_paths))
    for input_path, target_path in zip(input_img_paths[:10],
    target_img_paths[:10]):
    print(input_path, "|", target_path)
```

Code 4.5 Charger les chemins pour l'image d'entrée ainsi que pour le masque cible

```
    from IPython.display import Image, display
    from tensorflow.keras.preprocessing.image import load_img
    import PIL
    from PIL import ImageOps

    # Display input image #7
    display(Image(filename=input_img_paths[9]))

    # Display auto-contrast version of corresponding target
    (per-pixel categories)
    img = PIL.ImageOps.autocontrast(load_img(target_img_paths[9]))
    display(img)
```

Configurer le générateur de données

Plutôt que d'utiliser un générateur de données classique, j'ai choisi d'exploiter la classe **Sequence de Keras**, qui est plus fiable lorsqu'on fait du multitraitement. L'avantage est qu'un même échantillon n'est utilisé qu'une seule fois par époque, ce qui évite les répétitions ou des traitements inutiles. Pour cela, j'ai créé une classe personnalisée, *SegmentationDataset* **??**, qui hérite de *keras.utils.Sequence*. Son rôle est de charger automatiquement les images et leurs masques, puis de les transformer en lots prêts à être envoyés au modèle. La classe est composée de trois parties : une fonction d'initialisation qui définit la taille des images, la taille des lots et les chemins d'accès aux fichiers ; une fonction qui calcule combien de lots on peut générer ; et enfin une fonction principale (__getitem__) qui prépare les données. Concrètement, cette dernière lit les images d'entrée en couleur, les redimensionne et les normalise entre 0 et 1, puis fait de même pour les masques de segmentation en niveaux de gris. À la fin, elle retourne deux tableaux NumPy (x et y) parfaitement adaptés

à l'entraînement d'un modèle de segmentation binaire.

```
    from tensorflow import keras
    import numpy as np
    from tensorflow.keras.preprocessing.image import load_img,
    img_to_array

    class SegmentationDataset(keras.utils.Sequence):
    """Keras Sequence for binary segmentation tasks."""

    def __init__(self, batch_size, img_size, input_img_paths,
    target_img_paths,**kwargs):
    super().__init__(**kwargs)
    self.batch_size = batch_size
    self.img_size = img_size
    self.input_img_paths = input_img_paths
    self.target_img_paths = target_img_paths

    def __len__(self):
    return len(self.target_img_paths) // self.batch_size

    def __getitem__(self, idx):
    i = idx * self.batch_size
    batch_input_img_paths = self.input_img_paths[i: i +
    self.batch_size]
    batch_target_img_paths = self.target_img_paths[i: i +
    self.batch_size]

    # Inputs
    x = np.zeros((self.batch_size,) + self.img_size + (3,),
    dtype="float32")
    for j, path in enumerate(batch_input_img_paths):
    img = load_img(path, target_size=self.img_size)
    img_array = img_to_array(img) / 255.0  # Normalize to [0, 1]
    x[j] = img_array

    # Targets
    y = np.zeros((self.batch_size,) + self.img_size + (1,),
    dtype="float32")
    for j, path in enumerate(batch_target_img_paths):
    img = load_img(path, target_size=self.img_size,
    color_mode="grayscale")
    img_array = img_to_array(img) / 255.0  # Normalize 255     1
    y[j] = img_array  # shape (H, W, 1), values 0.0 or 1.0

    return x, y
```

Code 4.6 Classe Keras Sequence pour charger images et masques

Il faut organiser nos données pour l'entraînement. Le code commence par définir les proportions de validation et de test (ici 10% pour chacun), puis mélange les chemins des images et des masques de manière synchronisée grâce à une graine aléatoire fixe pour garantir la reproductibilité. Ensuite, le jeu de données complet est découpé en trois sous-ensembles : un jeu d'entraînement, un jeu de validation et un jeu de test. Cette séparation est essentielle pour entraîner correctement le modèle, valider ses performances pendant l'apprentissage et enfin l'évaluer objectivement sur des données encore jamais

vues. Pour finir, trois générateurs SegmentationDataset sont instanciés à partir de ces sous-ensembles, ce qui permet de préparer automatiquement les lots d'images et masques adaptés à chaque phase du cycle d'entraînement.

```
import random

# Set split ratios
val_ratio = 0.1
test_ratio = 0.1  # 10% for test, adjust as needed

# Shuffle with fixed seed for reproducibility
random.seed(1337)
combined = list(zip(input_img_paths, target_img_paths))
random.shuffle(combined)
input_img_paths, target_img_paths = zip(*combined)

# Calculate sizes
total_samples = len(input_img_paths)
val_samples = int(total_samples * val_ratio)
test_samples = int(total_samples * test_ratio)

# Split into train / val / test
train_input_img_paths = input_img_paths[:-(val_samples + test_samples)]
train_target_img_paths = target_img_paths[:-(val_samples + test_samples)]

val_input_img_paths = input_img_paths[-(val_samples + test_samples):-test_samples]
val_target_img_paths = target_img_paths[-(val_samples + test_samples):-test_samples]

test_input_img_paths = input_img_paths[-test_samples:]
test_target_img_paths = target_img_paths[-test_samples:]


print(f"Train samples: {len(train_input_img_paths)}")
print(f"Val samples:   {len(val_input_img_paths)}")
print(f"Test samples:  {len(test_input_img_paths)}")

# Instantiate data generators
train_gen = SegmentationDataset(batch_size, img_size, train_input_img_paths, train_target_img_paths)
val_gen = SegmentationDataset(batch_size, img_size, val_input_img_paths, val_target_img_paths)
test_gen = SegmentationDataset(batch_size, img_size, test_input_img_paths, test_target_img_paths)
```

4.5 Entraîner le modèle

4.5.1 Fonctions de perte

Dans le cadre de la segmentation binaire, il est essentiel de choisir une fonction de perte adaptée afin d'évaluer correctement la performance du modèle. Dans notre implémentation, nous avons utilisé une combinaison entre l'entropie croisée binaire (*Binary Crossentropy*) et le coefficient de Dice.

L'**entropie croisée** [**?**] est une mesure en apprentissage automatique et en théorie de l'information qui indique à quel pixel une distribution de probabilité prédite q et différente de la distribution réelle p.

$$H(p, q) = -\sum_{i} p(i) \log(q(i))$$

- $p(i)$: probabilité réelle de la classe
- $q(i)$: probabilité prédite de la classe

Dans notre projet on a deux classe et aussi on va faire la somme de $H_{pixel}(p, q)$ sur tous les pixels.

$$\mathcal{L}_{BCE}(y_{true}, y_{pred}) = -\sum_{i,j \in D} y_{true}[i, j] \log(y_{pred}[i, j]) + (1 - y_{true}[i, j]) \log(1 - y_{pred}[i, j])$$

- $D_1 = \{\, k \in \mathbb{N} \mid 1 \leq k \leq Image.shpae[0] \,\}$
- $D_2 = \{\, k \in \mathbb{N} \mid 1 \leq k \leq Image.shpae[1] \,\}$
- $D = D_1 \times D_2$

Le **coefficient de Dice** [**?**, **?**] est une métrique couramment utilisée en segmentation d'images car il mesure la similarité entre les masques prédits et les masques réels. Il est défini comme suit :

$$Dice(y_{true}, y_{pred}) = \frac{2 \times |y_{true} \cap y_{pred}| + \epsilon}{|y_{true}| + |y_{pred}| + \epsilon}$$

où ϵ est une constante de lissage qui permet d'éviter la division par zéro. La fonction de perte associée est alors :

$$\mathcal{L}_{Dice}(y_{true}, y_{pred}) = 1 - Dice(y_{true}, y_{pred})$$

Afin de tirer parti à la fois de la robustesse de l'entropie croisée binaire (qui pénalise les mauvaises prédictions pixel par pixel) et de la capacité du Dice à capturer le recouvrement global, nous utilisons une fonction de perte combinée :

$$\mathcal{L}_{combinée}(y_{true}, y_{pred}) = \mathcal{L}_{BCE}(y_{true}, y_{pred}) + \mathcal{L}_{Dice}(y_{true}, y_{pred})$$

Cette combinaison permet d'obtenir un entraînement plus stable et de meilleures performances sur des tâches de segmentation médicale et d'images complexes.

```
    # Dice Coefficient implementation
    def dice_coef(y_true, y_pred, smooth=1):
    intersection = K.sum(y_true * y_pred, axis=[1,2,3])
    union = K.sum(y_true, axis=[1,2,3]) + K.sum(y_pred, axis=[1,2,3])
    return K.mean((2. * intersection + smooth)/(union + smooth),
   axis=0)

    def dice_loss(y_true, y_pred):
    return 1 - dice_coef(y_true, y_pred)

    # Combined loss (binary crossentropy + dice)
    def combined_loss(y_true, y_pred):
    return binary_crossentropy(y_true, y_pred) + dice_loss(y_true,
   y_pred)
```

4.5.2 Optimiseur Adam et compilation du modèle

Pour entraîner notre réseau de neurones, nous avons choisi l'optimiseur **Adam (Adaptive Moment Estimation)** [**?**]. Adam est un algorithme de descente de gradient stochastique qui combine les avantages de deux méthodes populaires : AdaGrad (adaptation du taux d'apprentissage) et RMSProp (moyenne pondérée des gradients). Il ajuste dynamiquement le taux d'apprentissage pour chaque paramètre du modèle, ce qui favorise une convergence rapide et efficace.

Dans notre cas, nous avons fixé un taux d'apprentissage (*learning rate*) de 1×10^{-4} afin de garantir une descente progressive et stable de la fonction de perte.

La compilation du modèle se fait comme suit :

```
    model.compile(
    optimizer=Adam(learning_rate=1e-4),
    loss=combined_loss,
    metrics=['accuracy', 'dice_coef']
    )
```

Ainsi, notre modèle est entraîné avec la fonction de perte combinée, optimisé par Adam, et évalué grâce à deux métriques principales : la précision (*accuracy*) et le coefficient de Dice.

4.5.3 callbacks

TensorFlow et Keras intègrent des fonctionnalités avancées comme l'EarlyStopping, qui arrête l'entraînement lorsque le modèle cesse de s'améliorer, ou encore les Model Checkpoints, qui sauvegardent les meilleurs poids du modèle au cours de l'entraînement. Ces outils sont particulièrement utiles pour optimiser l'entraînement et s'assurer que le modèle ne surapprend pas.

```
    # Callbacks setup
    os.makedirs('/content/drive/MyDrive/SI/callbacks', exist_ok=True)
```

```
    callbacks = [
    ModelCheckpoint(
    '/content/drive/MyDrive/SI/callbacks/model_2023_only.keras',
    monitor='val_dice_coef',
    mode='max',
    save_best_only=True,
    verbose=1
    ),
    EarlyStopping(
    monitor='val_dice_coef',
    patience=20,
    mode='max',
    restore_best_weights=True
    ),
    ReduceLROnPlateau(
    monitor='val_loss',
    factor=0.1,
    patience=5,
    min_lr=1e-6,
    verbose=1
    ),
    CSVLogger('/content/drive/MyDrive/SI/callbacks/training_log.csv')
    ]
```

4.5.4 Apprentissage du Modèle et Résultats

Apprentissage du Modèle

Le modèle a été entraîné sur 30 époques (epochs) en utilisant une architecture de type U-Net. L'optimisation a été réalisée avec l'optimiseur Adam avec un taux d'apprentissage (learning rate) de valeur 10^{-4} et une taille de lot (batch size) de taille 8.

L'objectif de la phase d'apprentissage était de minimiser la fonction de perte combinée, entre l'Entropie Croisée Binaire (BCE) et la perte Dice, particulièrement adaptée aux tâches de segmentation car elle optimise directement la précision du recouvrement entre les régions.

Pour éviter le surapprentissage (overfitting) et garantir la capacité de généralisation du modèle, le jeu de données a été divisé en un ensemble d'apprentissage (80

```
    # Train the model, validating at the end of each epoch.
    epochs = 30
    model.fit(train_gen, epochs=epochs, validation_data=val_gen,
    callbacks=callbacks)
```

Résultats et Analyse de la Convergence

L'évolution des métriques durant l'apprentissage est présentée dans la **??**. Celle-ci montre la progression conjointe de la fonction de perte (Loss) et du coefficient de Dice sur les

ensembles **d'entraînement (en bleu)** et de **validation (en rouge)** au cours des 30 époques.

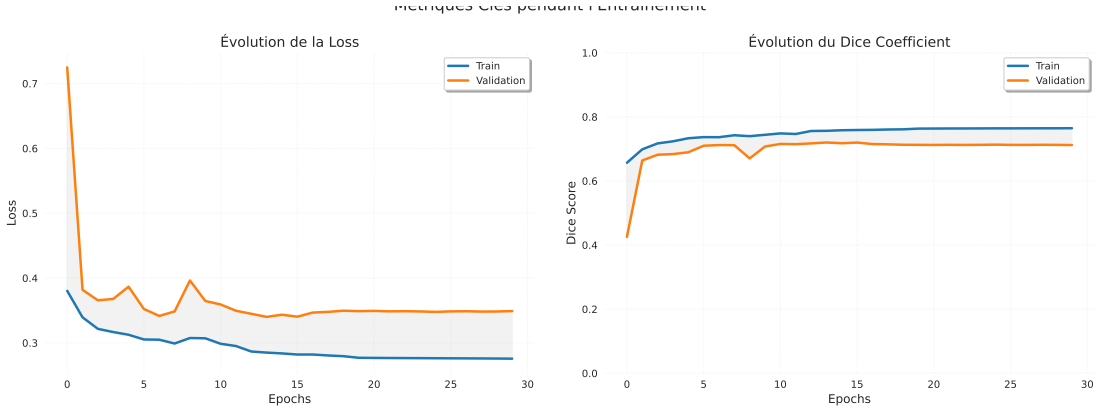


Figure 4.3 – Courbes d'Apprentissage.Évolution de la perte (Loss) et du score Dice sur les ensembles d'entraînement (bleu) et de validation (rouge) en fonction des époques.

L'analyse de ces courbes révèle une **convergence rapide et efficace** du modèle. On observe les tendances suivantes :

- **Perte (Loss) :** Les courbes de perte, tant pour l'entraînement (bleu) que pour la validation (rouge), affichent une **diminution forte et régulière** au cours des 15 premières époques. Les deux courbes suivent une trajectoire très similaire avant de se stabiliser. Comme attendu, la perte finale minimale sur l'ensemble d'entraînement (**0.276**) est légèrement inférieure à celle de la validation (**0.340**). Cet écart minime est normal et indique que le modèle n'a pas mémorisé les données d'entraînement mais a plutôt appris à généraliser.
- **Coefficient de Dice :** En corrélation directe avec la chute de la perte, les scores Dice sur l'entraînement (bleu) et la validation (rouge) **augmentent de manière significative**. Les deux courbes progressent ensemble avant d'atteindre un plateau. La performance maximale sur l'ensemble d'entraînement atteint **0,765**, tandis que le score sur la validation se stabilise à une excellente valeur de **0,720**. La proximité de ces deux valeurs finales confirme la **très bonne capacité de généralisation** du modèle.

Le comportement étroitement superposé des métriques d'entraînement et de validation, avec un écart final raisonnable, est le signe d'un **apprentissage sain et robuste**. L'absence de divergence claire entre les courbes rouge et bleue atteste de **l'absence de surapprentissage** notable.

Table 4.1 – Résultats quantitatifs finaux après 30 époques.

Métrique	**Entraînement**	**Validation**
Coefficient de Dice (max)	0.765	0.720
Perte (min)	0.276	0.340

Ces résultats démontrent l'efficacité de la configuration d'apprentissage, le modèle ayant atteint une performance élevée et équilibrée sur les deux ensembles de données. Le score

Dice de validation de **0,72** constitue une base solide pour la segmentation sur de nouvelles données.

Chapitre 5

Analyse Comparative des Performances : U-Net versus OpenCV

5.1 Métriques d'évaluation : Dice et IoU

Pour évaluer la qualité d'une segmentation, il ne suffit pas de « regarder à l'œil nu » : il faut des mesures objectives permettant de comparer la prédiction d'un modèle avec la vérité terrain (*ground truth*). Deux métriques très utilisées sont le **coefficient de Dice** et l'**Intersection over Union (IoU)**. Elles quantifient toutes les deux le degré de recouvrement entre ce qui est prévu et ce qui est réellement présent. [**?**, **?**]

5.1.1 Coefficient de Dice

Le coefficient de Dice, ou *Dice Similarity Coefficient (DSC)*, peut être vu comme une mesure de similarité. Il compare la taille de l'intersection entre deux ensembles A (la vérité terrain) et B (la prédiction), par rapport à leur taille combinée.

$$Dice(A, B) = \frac{2|A \cap B|}{|A| + |B|}$$

En pratique :

- Une valeur de **1** signifie une correspondance parfaite (toutes les prédictions sont correctes).
- Une valeur de **0** signifie qu'il n'y a aucun recouvrement.

On peut voir le Dice comme une sorte de « pourcentage d'accord » entre les deux masques.

5.1.2 Intersection over Union (IoU)

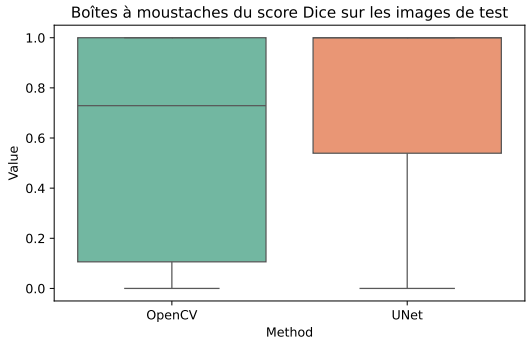
L'IoU [?], aussi appelée *Jaccard Index*, mesure le rapport entre l'intersection et l'union de A et B :

$$IoU(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

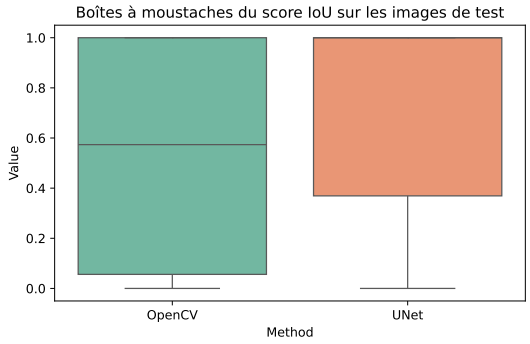
C'est une mesure intuitive : parmi tout ce qui est détecté **ou** présent dans la vérité terrain, quelle proportion est bien commune Dice ?

- IoU = **1** : recouvrement parfait.
- IoU = **0** : aucun recouvrement.

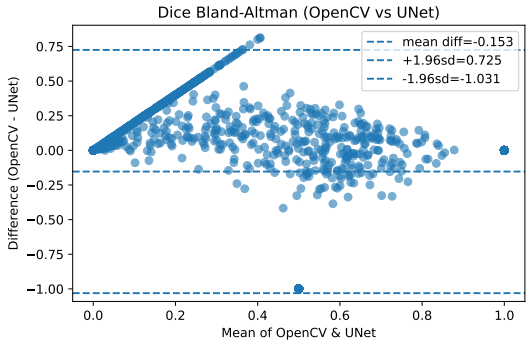
5.2 Évaluation Globale par Métriques de Segmentation



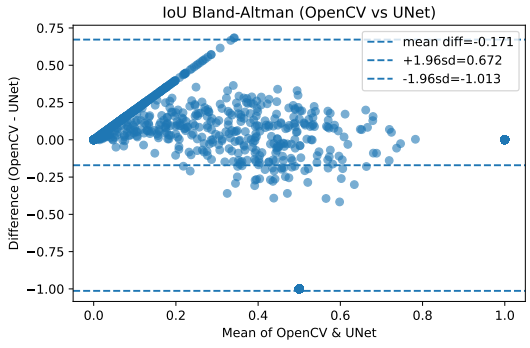
(a) Boîtes à moustaches des scores Dice



(b) Boîtes à moustaches des scores IoU



(c) Analyse Bland–Altman des scores Dice



(d) Analyse Bland–Altman des scores IoU

Figure 5.1 – Analyse comparative complète des performances de segmentation entre les approches U-Net et OpenCV.

5.2.1 Analyse Statistique des Métriques

Le tableau **??** présente les statistiques descriptives des métriques de segmentation pour les deux méthodes.

Table 5.1 – Résumé statistique des métriques IoU et Dice

Métrique	Méthode	Moyenne	Médiane	Écart-type
IoU	OpenCV	0.562	0.574	0.442
IoU	U-Net	0.733	1.000	0.409
Dice	OpenCV	0.601	0.729	0.426
Dice	U-Net	0.754	1.000	0.394

Performance absolue

- **U-Net** présente une performance nettement supérieure avec un score Dice moyen de **0,754** contre **0,601** pour OpenCV.
- La médiane des scores souligne également un écart marqué : **1,000** pour U-Net contre **0,729** pour OpenCV.
- L'amélioration relative de U-Net est d'environ **25 %** sur le score moyen, confirmant une meilleure capacité de généralisation.

Consistance des performances

- L'écart-type plus faible observé pour U-Net (**0,394**) indique une performance plus stable et homogène.
- OpenCV présente une variabilité élevée (**0,426**), traduisant une dépendance accrue aux conditions d'entrée et aux réglages.

5.2.2 Analyse par Boîtes à Moustaches

Les figures **??** et **??** montrent la distribution des scores de segmentation.

Distribution des scores

- **U-Net** : la majorité des scores sont concentrés dans la zone haute (valeurs supérieures à 0,8), traduisant une segmentation fiable sur l'ensemble du jeu de test.
- **OpenCV** : la distribution est plus étalée, avec des scores faibles sur plusieurs images, témoignant d'une sensibilité aux variations du contraste et du bruit.

Valeurs aberrantes

- OpenCV présente plusieurs valeurs extrêmes proches de 0, indiquant des échecs complets de segmentation.
- U-Net montre peu d'anomalies, démontrant une meilleure robustesse.

5.2.3 Analyse d'Accord par Bland–Altman

Les figures **??** et **??** illustrent les différences entre les deux méthodes à l'aide des diagrammes de Bland–Altman.

Biais systématique

- Pour le score Dice, la différence moyenne est de **-0,153** en faveur de U-Net.
- Pour le score IoU, le biais est similaire, avec une différence moyenne de **-0,171**.
- Les limites d'accord ([**-1,031 ; 0,725**] pour Dice) indiquent un faible accord global entre les deux approches.

Interprétation

- Ces résultats confirment que les performances d'U-Net surpassent systématiquement celles d'OpenCV, en particulier sur les images les plus complexes.
- Les différences observées sont significatives d'un point de vue pratique et visuel.

5.2.4 Synthèse et Discussion

Les analyses statistiques et graphiques montrent que :

- **U-Net** surpasse **OpenCV** en termes de précision, de stabilité et de cohérence des résultats.
- OpenCV conserve toutefois un intérêt pour les applications où la rapidité d'exécution et la simplicité de mise en œuvre priment sur la précision.

En conclusion, l'approche **U-Net** apparaît comme la méthode la plus adaptée pour les tâches de segmentation exigeant une haute fidélité et une bonne généralisation. Les différences observées dans les scores et les diagrammes d'accord valident statistiquement la supériorité de l'apprentissage profond sur les méthodes classiques basées sur des heuristiques.

5.3 le Choix de la Méthode

Le choix entre les approches **OpenCV** et **U-Net** dépend fortement du contexte applicatif et des contraintes associées, telles que les ressources matérielles disponibles, le niveau d'expertise requis et les exigences en matière de précision ou de rapidité. Le tableau **??** présente une synthèse comparative des deux approches selon différents critères d'application.

Analyse comparative

Table 5.2 – Comparaison et recommandations selon le contexte applicatif

Contexte	OpenCV	U-Net
Temps réel	Faible latence	× Calcul intensif
Précision	× Performance variable	Haute précision
Ressources	Faible coût de calcul	× Nécessite GPU
Expertise requise	Simple à implémenter	× Compétences ML nécessaires
Données d'entraînement	Non requises	× Données annotées nécessaires

- **OpenCV** est particulièrement adaptée aux environnements contraints en ressources (processeurs embarqués, systèmes légers ou applications temps réel). Son implémentation rapide et sa faible consommation de calcul en font un choix pertinent pour le prototypage ou les systèmes nécessitant une latence minimale.
- **U-Net**, fondée sur l'apprentissage profond, se distingue par une précision et une robustesse nettement supérieures. Elle est plus adaptée aux applications critiques où la qualité de la segmentation prime sur la vitesse d'exécution, notamment dans les domaines médical, industriel ou scientifique.

Synthèse décisionnelle

- Si la **rapidité** et la **simplicité** sont prioritaires : OpenCV demeure la solution la plus pragmatique.
- Si la **précision** et la **fiabilité** sont essentielles : U-Net constitue la méthode de référence.
- Dans une perspective d'**industrialisation**, une approche hybride pourrait être envisagée, combinant la détection rapide d'OpenCV à la précision de U-Net sur des régions d'intérêt ciblées.

Conclusion En définitive, le choix de la méthode doit être guidé par un compromis entre **performances**, **coût computationnel** et **contexte d'utilisation**. U-Net s'impose comme la solution de choix pour les applications nécessitant une segmentation fine et robuste, tandis qu'OpenCV conserve un intérêt pour les déploiements légers et réactifs. Une combinaison des deux approches représente une piste prometteuse pour concilier précision et efficacité dans les scénarios réels.

Annexes

Annexe A Dataset Description and Preparation
Annexe B Model Implementation using OpenCV
Annexe C Evaluation Metrics and Performance Analysis

Annexe A

Dataset Description and Preparation

A.1 Acquisition, traitement et exploitation des données satellitaires EUMETSAT

```python
import os
from satpy import Scene
from PIL import Image

# Region for cropping
MOROCCO_BBOX = {"lon_min": -20, "lon_max": 5, "lat_min": 20, "lat_max": 40.0}

def process_nat_to_image(nat_file_path: str, output_dir: str) -> bool:
    """
    Process a .nat file to create RGB image and remove the original file
    """
    try:
        # Extract timestamp from filename
        filename = os.path.basename(nat_file_path)
        parts = filename.split('-')

        # Find the timestamp part
        timestamp_part = None
        for part in parts:
            if part.startswith('202') and part.endswith('Z'):
                timestamp_part = part
                break

        if timestamp_part:
            # Convert to ISO format
            date_str = timestamp_part[:8]  # YYYYMMDD
            time_str = timestamp_part[8:14]  # HHMMSS

            iso_filename = f"{date_str[:4]}-{date_str[4:6]}-{date_str[6:8]}T{int(time_str[:2])+1}-00-00.0
        else:
```

```
            iso_filename = f"{datetime.datetime.utcnow().strftime('%Y-%m-%dT%H-%M-%S')}.000Z.png"

        output_path = os.path.join(output_dir, iso_filename)

        # Load and process the .nat file
        scn = Scene(filenames=[nat_file_path], reader="seviri_l1b_native")
        #scn.load(["IR_108", "IR_039", "IR_016", "VIS006", "WV_062", "WV_073"])
        scn.load(["convection"])
        # Crop to Morocco
        cropped_scn = scn.crop(ll_bbox=(MOROCCO_BBOX["lon_min"], MOROCCO_BBOX["lat_min"],
                                        MOROCCO_BBOX["lon_max"], MOROCCO_BBOX["lat_max"]))


        # Get the convection data
        cropped_scn.save_dataset('convection', filename=output_path)
        # Open saved file
        img = Image.open(output_path)

        # Rotate 180   and overwrite
        img.rotate(180).save(output_path)

        print(f"Convection image saved: {output_path}")
        return True
    except Exception as e:
        print(f"Error processing .nat file: {str(e)}")
        return False
```

Code A.1 ce fiche et admet une foncton qui resbemnsable sur transform les fiche nat to image png RGB de convection

```

import datetime
import json
import shutil
import zipfile
import os
from tqdm import tqdm
import numpy as np
import matplotlib.pyplot as plt
from src.api_eumdac import *
from src.nat_to_RGB_img import *
#
    ================================================================================
# CONFIGURATION
#
    ================================================================================

DATA_DIR = "data"


# Target hours for each year
YEAR_CONFIG = {
    2023: [3, 6, 9, 12, 15, 18, 21, 24],
```

```
    2024: [12, 15, 18]
}


# ============================================================================
# INITIALIZATION
# ============================================================================


def generate_dates():
    """Generate all dates to process based on year configuration"""
    dates = []
    for year, hours in YEAR_CONFIG.items():
        for month in range(1, 13):
            # Days in month calculation
            if month in [1, 3, 5, 7, 8, 10, 12]:
                days = 31
            elif month in [4, 6, 9, 11]:
                days = 30
            else:  # February
                days = 29 if (year % 4 == 0 and year % 100 != 0) or (year % 400 == 0) else 28

            for day in range(1, days + 1):
                dates.append((year, month, day, hours))

    return dates


# ============================================================================
# DOWNLOAD FUNCTIONS (FIXED)
# ============================================================================

def download_and_process_product(product, data_dir):
    """Download and process a single product using the correct method"""
    try:
        # Use product.open() to get the file stream and filename
        with product.open() as fsrc:
            # Get the filename from the stream
            filename = fsrc.name
            file_path = os.path.join(data_dir, filename)

            # Download the file
            with open(file_path, 'wb') as fdst:
                shutil.copyfileobj(fsrc, fdst)

            print(f' Downloaded: {filename}')

            # Check if it's a ZIP file
            if file_path.endswith('.zip'):
                print(f' Extracting: {filename}')

                # Extract the ZIP file
```

```
                with zipfile.ZipFile(file_path, 'r') as zip_ref:
                    zip_ref.extractall(data_dir)

                print(f' Extracted: {filename}')

                # Remove the ZIP file after extraction
                os.remove(file_path)
                print(f'     Removed ZIP: {filename}')

                # Process all .nat files that were extracted
                extracted_files = [f for f in os.listdir(data_dir)
    if f.endswith('.nat') and f not in filename]
                for nat_file in extracted_files:
                    nat_file_path = os.path.join(data_dir, nat_file)
                    process_nat_to_image(nat_file_path, data_dir)

            else:
                # If it's already a .nat file, process it directly
                process_nat_to_image(file_path, data_dir)

        return True

    except Exception as e:
        print(f" Error processing product: {str(e)}")
        # Clean up any partially downloaded files
        if 'file_path' in locals() and os.path.exists(file_path):
            os.remove(file_path)
        return False

#
    ================================================================================
# MAIN EXECUTION
#
    ================================================================================
def main():
    """Main download and processing workflow"""
    # Setup
    collection = setup_eumetsat()
    os.makedirs(DATA_DIR, exist_ok=True)

    dates = generate_dates()
    print(f" Processing {len(dates)} days across {len(YEAR_CONFIG)}
    years")
    print(f" Images will be saved to: {DATA_DIR}")
    print(f" Target region: {MOROCCO_BBOX}")

    total_downloaded = 0
    total_processed = 0

    # Main progress loop
    for year, month, day, target_hours in tqdm(dates, desc="Overall
    progress", unit="day"):
        date_str = f"{year}-{month:02d}-{day:02d}"

        try:
            # Process each target hour
            for hour in target_hours:
                # Adjust hour for 24-hour format
```

```
127
128
129                 # Create time window
130                 start = datetime.datetime(year, month, day, hour -
    1, 55)
131                 end = datetime.datetime(year, month, day, hour-1, 59)
132
133                 # Search for products
134                 products = list(collection.search(dtstart=start,
    dtend=end))
135
136                 if not products:
137                     continue
138
139                 # Download and process each product
140                 for product in products:
141                     if download_and_process_product(product,
    DATA_DIR):
142                         total_downloaded += 1
143                         total_processed += 1
144
145         except Exception as e:
146             print(f" Error processing {date_str}: {e}")
147             continue
148
149     # Final cleanup and summary
150     print(f"\n Processing completed!")
151     print(f" Total products downloaded: {total_downloaded}")
152     print(f"     Total images created: {total_processed}")
153
154     # Check for any remaining .nat files (should be none)
155     remaining_nat_files = [f for f in os.listdir(DATA_DIR) if
    f.endswith('.nat')]
156     if remaining_nat_files:
157         print(f"  Warning: {len(remaining_nat_files)} .nat files
    remaining (processing failed)")
158         for file in remaining_nat_files:
159             print(f"   - {file}")
160
161     # Show final image count
162     image_files = [f for f in os.listdir(DATA_DIR) if
    f.endswith('.png')]
163     print(f" Final image count: {len(image_files)}")
164     print(f" All images saved in: {DATA_DIR}")
165
166 if __name__ == "__main__":
167     main()
```

Code A.2 Téléchargement des images RGB de convection

A.2 Génération des images de segmentation à partir des annotations VGG

Une fois que nous avons obtenu le fichier JSON qui contenant toutes les informations de refion de convection qui obtenire par le site VGG Image Annotator , nous pouvons exécuter le code suivant. Ce script permet de générer automatiquement un dossier contenant toutes les images de segmentation correspondantes à chaque image annotée.

```
    from PIL import Image, ImageDraw
import json
import os

# === Paths ===
json_path = "annotations_via_2023.json"
image_folder = "data/image"
output_folder = "data/mask"
os.makedirs(output_folder, exist_ok=True)

# === Load JSON ===
with open(json_path, "r") as f:
    data = json.load(f)

# === Process images ===
for key, value in data.items():
    if value["filename"] == 'rdf':  # Stop condition
        break

    filename = value["filename"]
    image_path = os.path.join(image_folder, filename)

    if not os.path.exists(image_path):
        print(f" Image not found: {filename}")
        continue

    # Open image to get dimensions
    with Image.open(image_path) as img:
        width, height = img.size

    # Create binary mask (0=black background, 255=white regions)
    mask = Image.new("L", (width, height), 0)  # Start with black
    canvas
    draw = ImageDraw.Draw(mask)

    for region in value["regions"]:
        shape = region["shape_attributes"]
        points = list(zip(shape["all_points_x"],
    shape["all_points_y"]))
        draw.polygon(points, fill=255)  # FILL WITH WHITE (255)

    # Save mask (preserve original extension)
    name, ext = os.path.splitext(filename)
    mask_filename = f"mask_{name}{ext}"  # e.g., mask_image.jpg
    mask_path = os.path.join(output_folder, mask_filename)
    mask.save(mask_path)

    # Delete original image (optional)
```

```
47      os.remove(image_path)
48      print(f" Created mask for {filename}")
```

Code A.3 Génération des images de segmentation à partir des annotations VGG (JSON file)

Annexe B

Model Implementation using OpenCV

```python
import cv2
import numpy as np
import os

def segment_convection_colors(input_image_path, output_image_path):
    """
    Segments specific convection-related cloud colors (yellow, red, orange)
    from an input image and saves the result as a binary mask.

    Args:
        input_image_path (str): Path to the input image.
        output_image_path (str): Path where the output binary mask will be saved.

    Returns:
        bool: True if the image was processed successfully, False otherwise.
    """
    # Load the image
    image = cv2.imread(input_image_path)
    if image is None:
        print(f"Error: Could not read image {input_image_path}")
        return False

    # Convert the image to HSV color space (better for color segmentation)
    hsv = cv2.cvtColor(image, cv2.COLOR_BGR2HSV)

    # Define HSV ranges for different convection colors:

    # Yellow: Thick ice cloud with small crystals
    lower_yellow = np.array([20, 100, 100])
    upper_yellow = np.array([30, 255, 255])

    # Red: Thick ice cloud with large crystals
    lower_red1 = np.array([0, 100, 100])
    upper_red1 = np.array([10, 255, 255])
```

```
    lower_red2 = np.array([160, 100, 100])
    upper_red2 = np.array([180, 255, 255])

    # Orange: Intermediate clouds
    lower_orange = np.array([10, 100, 100])
    upper_orange = np.array([20, 255, 255])

    # Create binary masks for each color
    mask_yellow = cv2.inRange(hsv, lower_yellow, upper_yellow)
    mask_red = cv2.bitwise_or(
        cv2.inRange(hsv, lower_red1, upper_red1),
        cv2.inRange(hsv, lower_red2, upper_red2)
    )
    mask_orange = cv2.inRange(hsv, lower_orange, upper_orange)

    # Combine the masks (yellow OR red OR orange)
    mask_convection = cv2.bitwise_or(mask_yellow, mask_red)
    mask_convection = cv2.bitwise_or(mask_convection, mask_orange)

    # The result is already a binary mask (0 = black, 255 = white)
    binary_mask = mask_convection

    # Save the binary mask
    cv2.imwrite(output_image_path, binary_mask)
    return True


def process_folder(input_folder, output_folder):
    """
    Processes all supported image files in a folder, applies color segmentation,
    and saves the binary masks into the output folder.

    Args:
        input_folder (str): Path to the folder containing input images.
        output_folder (str): Path to the folder where results will be saved.
    """
    # Create the output folder if it doesn't exist
    os.makedirs(output_folder, exist_ok=True)

    # Supported image file formats
    supported_formats = ('.png', '.jpg', '.jpeg', '.bmp', '.tif', '.tiff')
    processed_count = 0

    # Iterate over all files in the input folder
    for filename in os.listdir(input_folder):
        if filename.lower().endswith(supported_formats):
            input_path = os.path.join(input_folder, filename)
            output_path = os.path.join(output_folder, f"mask_{filename}")

            if segment_convection_colors(input_path, output_path):
                print(f"Processed: {filename}")
                processed_count += 1
            else:
```

```
                print(f"Skipped: {filename}")

    print(f"\nProcessing completed. {processed_count} images
    processed.")


# Example usage
input_folder = "make data/2023/data-2023-abdellah"  # Path to your
    input folder
output_folder = "make data/2023/masks-opencv-2023-abdellah"  # Path
    to your output folder
os.makedirs(output_folder, exist_ok=True)
process_folder(input_folder, output_folder)
```

Code B.1 modele de segemntation automatique des images RGB de convection par OpenCV

Annexe C

Evaluation Metrics and Performance Analysis

Ce script **??** sera utilisé pour générer un fichier CSV, pour à comparer la performance d'OpenCV et de UNet.

```
import os, glob
import numpy as np
import cv2
import matplotlib.pyplot as plt
from PIL import Image
import tensorflow as tf
from tensorflow.keras.models import load_model
from tensorflow.keras.preprocessing.image import load_img,
    img_to_array


# --- Colab: monter Google Drive ---
from google.colab import drive
drive.mount('/content/drive')

# --- Chemins (adapte si besoin) ---
BASE = "/content/drive/MyDrive/SI"

DIR_IMAGES = os.path.join(BASE, "data_2023")            # Images RGB
DIR_MASK_EXPERT = os.path.join(BASE, "mask_via_2023")   # Masques
    expertise humaine
DIR_MASK_OPENCV = os.path.join(BASE, "mask_opencv_2023")# Masques
    OpenCV

DIR_OUT_CSV  = os.path.join(BASE, "metriques")
os.makedirs(DIR_OUT_FIGS, exist_ok=True)
os.makedirs(DIR_OUT_CSV,  exist_ok=True)
# Charger le mod le UNet
from tensorflow.keras.utils import register_keras_serializable
import tensorflow.keras.backend as K
@register_keras_serializable()
def dice_coef(y_true, y_pred, smooth=1):
    intersection = K.sum(y_true * y_pred, axis=[1,2,3])
    union = K.sum(y_true, axis=[1,2,3]) + K.sum(y_pred, axis=[1,2,3])
```

```
    return K.mean((2. * intersection + smooth) / (union + smooth),
    axis=0)

@register_keras_serializable()
def dice_loss(y_true, y_pred):
    return 1 - dice_coef(y_true, y_pred)

@register_keras_serializable()
def combined_loss(y_true, y_pred):
    return binary_crossentropy(y_true, y_pred) + dice_loss(y_true,
    y_pred)

model =
    load_model("/content/drive/MyDrive/SI/callbacks/model_2023_only.keras")

# R cup rer la taille d'entr e attendue (H, W, C)
input_shape = model.input_shape  # ex: (None, 160, 160, 3)
H, W, C = input_shape[1], input_shape[2], input_shape[3]

def read_rgb_resize(path, size=(256,256)):
    """Lit une image RGB et la redimensionne    size. Retour:
    tableau float32 [0,1]."""
    img = load_img(path, target_size=size)    # PIL      RGB
    arr = img_to_array(img) / 255.0           # [H,W,3] en [0,1]
    return arr

def read_mask_binary(path, size=(256,256)):
    """Lit un masque (png/jpg), convertit en binaire {0,1},
    redimensionn     size."""
    # On lit en niveaux de gris
    m = cv2.imread(path, cv2.IMREAD_GRAYSCALE)
    if m is None:
        return None
    m = cv2.resize(m, size, interpolation=cv2.INTER_NEAREST)
    # binarisation (127 seuil)
    m = (m > 127).astype(np.uint8)
    return m

def predict_unet_mask(img_rgb):
    """Pr dit le masque binaire {0,1} via le mod le,    partir
    d'une image RGB [0,1]."""
    x = np.expand_dims(img_rgb, axis=0)  # [1,H,W,3]
    pred = model.predict(x, verbose=0)[0]  # [H,W,1] ou [H,W] selon
    le mod le
    if pred.ndim == 3:
        pred = pred[..., 0]
    # seuillage    0.5
    m = (pred >= 0.5).astype(np.uint8)
    return m

def iou(mask_a, mask_b):
    """Intersection over Union pour masques binaires {0,1}."""
    inter = np.logical_and(mask_a==1, mask_b==1).sum()
    union = np.logical_or(mask_a==1, mask_b==1).sum()
    return inter / union if union > 0 else 1.0  # si tout vide, on
    met 1.0 par convention

def dice(mask_a, mask_b):
```

```
    """Dice coefficient pour masques binaires {0,1}."""
    inter = np.logical_and(mask_a==1, mask_b==1).sum()
    s = (mask_a==1).sum() + (mask_b==1).sum()
    return (2*inter) / s if s > 0 else 1.0


import csv
csv_path = os.path.join(DIR_OUT_CSV,
   "metrics_opencv_unet_vs_expert_2023.csv")
i = 0
with open(csv_path, "w", newline="") as f:
    writer = csv.writer(f)
    writer.writerow([
        "filename",
        "IoU_OpenCV_vs_Expert", "Dice_OpenCV_vs_Expert",
        "IoU_UNet_vs_Expert",  "Dice_UNet_vs_Expert"
    ])

    for ipath in img_paths:
        base = os.path.splitext(os.path.basename(ipath))[0]
        i += 1
        # Chercher masques expert & opencv
        cand_exts = [".png", ".jpg", ".jpeg"]
        m_expert_path = None
        m_opencv_path = None
        for ext in cand_exts:
            p1 = os.path.join(DIR_MASK_EXPERT, "mask_" + base + ext)
            p2 = os.path.join(DIR_MASK_OPENCV, "mask_" + base + ext)
            if m_expert_path is None and os.path.exists(p1):
                m_expert_path = p1
            if m_opencv_path is None and os.path.exists(p2):
                m_opencv_path = p2

        if (m_expert_path is None) or (m_opencv_path is None):
            print(f"[WARN] Masque manquant pour {base} ->
   expert:{m_expert_path} opencv:{m_opencv_path}")
            continue

        # Charger image + masques aux tailles du mod le
        img_rgb = read_rgb_resize(ipath, size=(W, H))
        m_expert = read_mask_binary(m_expert_path, size=(W, H))
        m_opencv = read_mask_binary(m_opencv_path, size=(W, H))

        if (m_expert is None) or (m_opencv is None):
            print(f"[WARN] Lecture masque   choue   pour {base}")
            continue

        # Pr dire avec UNet
        m_unet = predict_unet_mask(img_rgb)

        # Mesures OpenCV vs Expert
        iou_oe = iou(m_opencv, m_expert)
        dice_oe = dice(m_opencv, m_expert)

        # Mesures UNet vs Expert
        iou_ue = iou(m_unet, m_expert)
        dice_ue = dice(m_unet, m_expert)

```

```
            # Ligne CSV
            writer.writerow([base, iou_oe, dice_oe, acc_oe, cos_oe,
    iou_ue, dice_ue, acc_ue, cos_ue])

print("Termin . CSV:", csv_path)
```

Code C.1 Code Python utilisé pour générer un fichier CSV et comparer les performances d'OpenCV et de UNet